

Dominium Project Book

A human-readable synthesis of the current docs, archive context, and conversation evidence

Status: DERIVED

Authority Class: advisory_synthesis

Promotion Status: not_promoted

2026-05-29

Contents

1	Dominium Project Book	1
1.1	How to Read This Book	1
1.2	What Is Current Truth	1
1.3	What Is Historical Evidence	1
1.4	Reading Paths	1
2	Part I - The Project	2
2.1	1. Dominion in One View	2
2.2	2. Why the Project Exists	5
2.3	3. The Core Mental Model	8
2.4	4. Current Authority and Historical Context	11
3	Part II - The Architecture	15
3.1	5. The System Stack	15
3.2	6. Domino, Dominion, Runtime, Products, and Tools	18
3.3	7. Law, Capability, Refusal, and Evidence	22
3.4	8. Determinism, Replay, and Provenance	25
3.5	9. Contracts, Schemas, Packs, and Compatibility	28
4	Part III - Product and Operator Surfaces	32
4.1	10. Client, Server, Launcher, Setup, and Workbench	32
4.2	11. UI, Renderer, Native GUI, and Presentation Boundaries	36
4.3	12. AIDE, Codex, Automation, and Repo Governance	39
4.4	13. Documentation, Archive, and Conversation Corpus	42
5	Part IV - Simulation and World Model	46
5.1	14. Reality, Space, Time, and Scale	46
5.2	15. World Generation, Celestial Systems, and Domains	50
5.3	16. Civilization, Economy, Logistics, and Institutions	53
5.4	17. Content, Providers, Modding, and Data-Driven Extension	57
6	Part V - Decisions and Roadmap	61
6.1	18. What Is Settled	61

6.2	19. What Is Still Open	64
6.3	20. What Is Blocked	67
6.4	21. Contradictions and Drift	69
6.5	22. Prerequisites and Sequencing	72
6.6	23. Recommended Next Steps	75
7	Appendices	79
7.1	Appendix A - Decision Index	79
7.2	Appendix B - Open Questions Index	79
7.3	Appendix C - Source Notes Companion	80
7.4	Appendix D - Topic Index	80

Chapter 1

Dominium Project Book

A human-readable synthesis of the current docs, archive context, and conversation evidence

Status: DERIVED. Authority Class: advisory_synthesis. Promotion Status: not_promoted.

This book is a reader surface. It summarizes current documentation, archive context, and conversation evidence without promoting archive claims into current repo authority. Current truth still lives in canon, glossary, agent governance, contracts, schema law, current queue state, validated repo artifacts, and current docs according to the repository authority order.

1.1 How to Read This Book

Read the chapters as prose first. The source notes are a companion for audit and review, not a substitute for the explanation in the chapters. The book deliberately avoids visible evidence-card IDs and raw path trails in the main body so the reading experience is not a machine report.

1.2 What Is Current Truth

Current truth is the material backed by the repository's canonical and current authority surfaces. The book gives that material priority whenever it conflicts with generated summaries, old archive documents, or conversation evidence.

1.3 What Is Historical Evidence

Historical evidence includes archived docs, generated corpus reports, reader pages, and conversation-derived synthesis. It is useful because it preserves design intent, old decisions, unresolved questions, and contradictions. It is not current authority unless a later explicit task promotes it.

1.4 Reading Paths

For a short orientation, read Part I and Chapter 23. For architecture, read Parts II and III. For simulation intent, read Part IV. For planning and review work, read Part V and then the source notes companion.

The conversation archive is represented across the prose through source-derived themes and through the source notes companion. It is not printed as transcripts or path lists in the main chapters.

Chapter 2

Part I - The Project

2.1 1. Dominium in One View

Dominium is easiest to understand as a layered project: Domino supplies the deterministic substrate, while Dominium gives that substrate product meaning, domain law, authored content, and operator surfaces.

The current repository frames the project as lawful simulation, explicit refusal, and evidence-backed execution rather than a conventional game executable.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Dominium is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. Dominium/Domino is a deterministic world runtime and law-governed simulation OS.

A second current-thread is the project's insistence that roles remain explicit. Runtime behavior MUST resolve from profile data ('ExperienceProfile', 'LawProfile', 'ParameterBundle', optional 'MissionSpec' constraints). All runtime-impacting schemas MUST declare 'schema id', 'schema version', and 'stability'. Deprecated terms must not be introduced in new runtime logic. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints.

Current documentation also narrows the interpretation of the theme. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. There are no game modes; behavior is law + capability + policy. Dominium is not a monolithic game executable, a renderer-owned simulation, a traditional mode-flag game, a silent fallback system, or a place where generated output becomes source truth by convenience. Travel scheduling, replay, and law decisions become nondeterministic.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout. When behavior or compatibility meaning changes, update the relevant contract surface and proof in the same task. Packs provide capabilities, data, and assets; executables do not embed content. Dependency direction is authoritative and must not be inverted.

The conversation archive stretches that identity toward a larger product ecosystem, with Workbench, launchers, world simulation, and future product surfaces treated as projections over the same substrate.

The conversation corpus adds more texture. Domino is the reusable deterministic substrate: execution or-

dering, storage, replay, ABI-facing boundaries, and engine mechanisms. Archived conversations consistently treat Domino as reusable infrastructure rather than a one-off game binary. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. Which ideas are final decisions and which are recommendations? Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Reader Brief - Dominion Canon, Repository Alignment, and Portability Doctrine. SELF-AUDIT AND REVISION - Dominion Canon, Repository Alignment, and Portability Doctrine. Separated repo docs, schemas, code, and tests in maturity claims. Reliability rating: 4/5 with explicit caveats.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process.

Other decisions remain tentative because the source material records review posture rather than final promotion. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Which ideas are final decisions and which are recommendations? Calendars are pure renderers and never affect physics. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays.

The governing constraint is practical rather than cosmetic. The evaluation step where AuthorityContext is

validated before Process execution. Do not perform silent migrations or compatibility reinterpretation. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Scope: single-source dependency map and forbidden edges for Dominion/Domino.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. Game code depends on logical components, not layout or memory order. Scope: canonical statement of identity and intent. Do not infer ownership from similarly named roots.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed.

The important boundary is that ambition does not equal authority. Historical sources can explain why the system is shaped this way, but they do not authorize broad implementation by themselves.

For dominium in one view, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around identity, current project picture, Domino versus Dominion points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use dominium in one view as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for identity, current project picture, Domino versus Dominion, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Inconsistent clients, shard drift, and audit failure. Crosses unresolved or quarantined ownership areas. Refusal and defer semantics are part of determinism and replay guarantees. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. 'archive/': historical, superseded, quarantined, or provenance-retained. The current repo should not be overstated as a complete runtime implementation of all v1 goals. 7. The current code is promising but not yet the full vision. It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired. This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the

implied implementation step. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust. Old repo snapshot/project attachments, not yet verified. VM-only vs native plugin policy is unresolved. SDK locking was treated as mandatory. The warning was that allowing a newer Windows SDK to leak into old-floor builds can create symbol/runtime-loader failures and hidden dependency creep.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation. A 14-prompt Codex implementation roadmap exists.

Read this first as the compact mental model for the rest of the book: substrate first, domain meaning second, products and tools downstream.

2.2 2. Why the Project Exists

The project exists because it wants emergent social and material complexity to come from lawful process rather than from scripted outcomes.

Current orientation material names invention, production, logistics, economics, settlement, communication, trust, and institutional power as recurring ends of the simulation model.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. Dominion/Domino is a deterministic world runtime and law-governed simulation OS. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints.

A second current-thread is the project's insistence that roles remain explicit. Dominion is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout. When behavior or compatibility meaning changes, update the relevant contract surface and proof in the same task.

Current documentation also narrows the interpretation of the theme. Packs provide capabilities, data, and assets; executables do not embed content. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. A multi-scale reality model with explicit existence/refinement and visitability. Micro detail is refined only when required; macro existence is valid.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Dominion is not a monolithic game executable, a renderer-owned simulation, a traditional mode-flag game, a silent fallback system, or a place where generated output becomes source truth by convenience. Current partially aligned to the Constitutional Architecture and release-freeze documentation set. The simulation kernel is the source of truth; perception layers are derived. Objective snapshots record authoritative state; subjective snapshots record views.

Conversation material expands the same idea into seed civilizations, institutions, astronomical scale, authored

worlds, and operator tooling.

The conversation corpus adds more texture. Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical. The main outcome was a practical doctrine: use commercial engines for what they are good at, but do not delegate the canonical universe state to them. Determinism, resource accounting, fog-of-war authority, sparse planet edits, region handoff. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The strongest creative premise was the robotic seed-civilisation idea. Humans in the past built a robotic mothership, sent it to another solar system, and tasked it with landing on a habitable planet and manufacturing robots to restart civilisation. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. Backend and serialization audits are still required. What is Domino exactly? Should Unreal be first frontend? Which language/math stack should DominionSim use? What terrain model is best? How exact must collapsed simulation be? What first MMO scale target is realistic?

Several decisions are folded into this topic rather than standing apart from it. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse.

Other decisions remain tentative because the source material records review posture rather than final promotion. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto.

The decision pattern that matters most is the distinction between a design direction that has been preserved

and a rule that has actually become binding. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. Support canonical surface plus multiple surface instances for shards/mirrors.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Do not perform silent migrations or compatibility reinterpretation. Scope: single-source dependency map and forbidden edges for Dominium/Domino. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not infer ownership from similarly named roots. The main tradeoff was between ambition and discipline. Dominium's vision is huge: universe-scale exploration, arbitrary invention, civilization simulation, institutions, knowledge propagation, and moddable alternate realities. But the repo and doctrine. The central proposal from that first phase was that Dominium should follow a kernel-first deterministic simulation route. In plain terms, this means starting with the smallest reliable simulation core: time, deterministic math, random number. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. The mothership solves several design problems at once. It explains why players have a HUD, blueprints, recipes, construction guidance, and advanced technical knowledge. Players do not need to reinvent CPUs, railway gauges, smelting, computing, or. The main outcome is not that Dominium needs a conceptual rewrite. The outcome is that Dominium's existing model is directionally correct, but needs boundary hardening: audit all DSYS backend timers, freeze ACT units and serialization semantics, prevent. How exactly to handle 64-bit integers under C89 constraints?

The tension is scope: the purpose is broad, while the current queue permits only narrow, governed work.

For why the project exists, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around motivation, long-horizon ambition, emergent systems points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use why the project exists as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for motivation, long-horizon ambition, emergent systems, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate. Crosses unresolved or quarantined ownership areas. 'archive/': historical, superseded, quarantined, or provenance-retained. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The current repo should not be overstated as a complete runtime implementation of all v1 goals. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. Proper time matters for local clocks, aging, and relativistic effects, but making it the basis of civil calendars would create frame-dependent disagreements and log problems. It was rejected as authoritative calendar time. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. 7. The current code is promising but not yet the full vision. It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation. The discussion then mapped these ideas onto Dominion's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX, BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We.

The practical reading is that Dominion's ambition is durable, but every concrete feature still needs current authority, contracts, and validation before becoming implementation.

2.3 3. The Core Mental Model

The core mental model is a separation of truth, perception, rendering, and operation.

Canon and architecture documents make truth authoritative, perception a filtered view, rendering a presentation layer, and mutation something that can happen only through lawful deterministic process.

The current material supports that picture in several ways. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Dominion/Domino is a deterministic world runtime and law-governed simulation OS. Travel scheduling, replay, and law decisions become nondeterministic. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence.

A second current-thread is the project's insistence that roles remain explicit. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. A multi-scale reality model with explicit existence/refinement and visitability. Authoritative logic is deterministic and law-gated.

Current documentation also narrows the interpretation of the theme. Micro detail is refined only when required; macro existence is valid. Continue documentation and evidence updates that do not bypass Foundation laws. Replays diverge, cross-shard reconciliation fails, and saves become invalid. It is built to host multi-scale universes with explicit existence, travel,.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. It is not acceptable to fabricate existence or resources. Any state transition that cannot be expressed under deterministic ordering, legal authority context, and auditable commit semantics is invalid. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints. Dominium is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition.

Older conversations repeatedly test that model against Workbench, user interfaces, providers, packs, world generation, and future editor workflows.

The conversation corpus adds more texture. The main outcome was a practical doctrine: use commercial engines for what they are good at, but do not delegate the canonical universe state to them. Determinism, resource accounting, fog-of-war authority, sparse planet edits, region handoff. The central conclusion was that no commercial engine can provide this whole stack out of the box. Unreal Engine can be useful, especially as a renderer, editor, visual client, construction-preview tool, and asset pipeline. UE6 may eventually be useful if. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. What is Domino exactly? Should Unreal be first frontend? Which language/math stack should DominiumSim use? What terrain model is best? How exact must collapsed simulation be? What first MMO scale target is realistic? The final user-confirmed world model uses a toroidal horizontal SurfaceGrid of 2^{24} metres per axis, one vertical Page of 4096 metres, 16 metre cubic chunks, runtime Q16.16 coordinates with 8-bit Segment addressing, and Q4.12 chunk-local save.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. One canonical surface saved per planet for now. Chunks must not own canonical simulation state. The practical purpose is simple: a future reader should be able to open this one file and quickly understand the substance of the conversation without needing to reread the whole transcript. It is not a replacement for the structured package; it is the. Calendars are pure renderers and never affect physics.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Which decisions are confirmed versus recommendations? Review decisions with uncertainty labels, then formalise the vertical slice.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system

first. Domino should store the true universe as seeds, rules, fields, sparse. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. Calendars are pure renderers and never affect physics.

Other decisions remain tentative because the source material records review posture rather than final promotion. Domain permissions + existence/refinement - visitability decision. Which decisions are confirmed versus recommendations? Review decisions with uncertainty labels, then formalise the vertical slice. Produce audit records for provenance and law decisions.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Series Scope: repo-structure discovery and design Series Role: authoritative pre-layout validation, decision, and packaging checkpoint for the completed repo-structure series; downstream of stronger canon, the six repo-structure packets, post-Zeta. Must not inject nondeterministic decisions into protocol acceptance/refusal logic. ROI expand/collapse decisions therefore depend on deterministic tick order, not wall-clock cadence. Proof hash chains present (emission, field, exposure, compliance, decision/degrade).

The governing constraint is practical rather than cosmetic. Save/replay integrity depends on identical macro transitions. Do not perform silent migrations or compatibility reinterpretation. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Scope: single-source dependency map and forbidden edges for Dominion/Domino.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Scope: canonical statement of identity and intent. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed. Game code depends on logical components, not layout or memory order. Do not infer ownership from similarly named roots.

The model becomes fragile whenever a convenience layer is allowed to correct truth, hide refusal, or store authoritative state because it happens to be close to the user.

For the core mental model, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around truth/perception/render separation, process-only mutation, refusal points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use the core mental model as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for truth/perception/render separation, process-only mutation, refusal, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant

knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. Inconsistent clients, shard drift, and audit failure. Crosses unresolved or quarantined ownership areas. 'archive/': historical, superseded, quarantined, or provenance-retained.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a. SDK locking was treated as mandatory. The warning was that allowing a newer Windows SDK to leak into old-floor builds can create symbol/runtime-loader failures and hidden dependency creep. Collapsed tiles are represented by macro capsules and are removed from the tile cache. Expand regenerates tiles deterministically from the provider chain.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed.

The reader should treat this as the book's organizing rule: every future surface must say whether it owns truth, observes truth, presents truth, or merely helps operate the repository.

2.4 4. Current Authority and Historical Context

The documentation corpus is not flat. It has authority layers, and the book only makes sense if those layers are kept visible.

Current repo truth starts with canon, glossary, agent governance, active planning and contracts, queue state, and validated repo artifacts.

The current material supports that picture in several ways. Run 'AIDE-WORKFLOW-LAW-01' next if the queue remains reconciled after. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice.

A second current-thread is the project's insistence that roles remain explicit. Dependency direction is authoritative and must not be inverted. Law admission gates sit before scheduling, before execution, and before commit. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Scope: hard invariants for all systems, documentation, and data.

Current documentation also narrows the interpretation of the theme. Reachability lies, bypassed costs, and broken domain law. Invariant: No authority without capability and law. Invariant: Authority gates actions

only, never visibility (AUTH3-AUTH-001). Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003).

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Invariant: Authority upgrades/downgrades do not mutate state (AUTH3-UPGRADE-007). Invariant: Saves are tagged by authority scope (AUTH3-SAVE-008). Dominion/Domino is a deterministic world runtime and law-governed simulation OS. A law-governed simulation OS where authority is data-defined.

The archive and conversation corpus preserve useful design memory, old plans, contradictions, and proposed future directions.

The conversation corpus adds more texture. Any claim touching canon, glossary, authority order, contracts, schema law, current queue, release, implementation, or blocked scope. The conversations strongly prefer a rich Workbench and eventual editor/operator surfaces. Current authority is narrower: broad Workbench UI, renderer implementation, and native GUI remain blocked. Therefore synthesis may describe intent, but follow-up. Do not patch canon, contracts, schema, implementation, release, or queue files from this atlas. This atlas is a reader map. It is not canon and does not promote archived conversation claims.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. This map combines current repo orientation with conversation-derived design intent. When they differ, current repo authority wins. This reader page is derived from archived conversation material. It cannot override canon, contracts, schema law, current queue state, live repo structure, or validated repo artifacts. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?". Reader Brief - Dominion Canon, Repository Alignment, and Portability Doctrine.

The archive also records how the project repeatedly returned to the same pressure points from different directions. SELF-AUDIT AND REVISION - Dominion Canon, Repository Alignment, and Portability Doctrine. Separated repo docs, schemas, code, and tests in maturity claims. Reliability rating: 4/5 with explicit caveats. How should public API/ABI boundaries be written?

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30.

Other decisions remain tentative because the source material records review posture rather than final promotion. Support canonical surface plus multiple surface instances for shards/mirrors. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?".

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Domain permissions + existence/refinement - visitability decision. Non-canonical command accepted as legacy path. From law decisions and recorded in audit logs. Parallel execution is a backend choice, not a gameplay decision. Different.

The governing constraint is practical rather than cosmetic. Scope: single-source dependency map and forbidden edges for Dominion/Domino. Scope: canonical statement of identity and intent. An Assembly capable of submitting Intents under AuthorityContext and LawProfile constraints. Foundation Lock does not mean full CTest is green, every compatibility corpus exists, every provider is implemented, every runtime trust rule is enforced, Workbench UI exists, or broad feature work is open.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Keep Queue B hardening planned but not a substitute for the blocker repair. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. "Convenient" implicit behavior is not allowed. Client and renderer project data and issue intents; they do not author authoritative outcomes. A term or pattern forbidden for use in code identifiers. 'FOUNDATION-CLOSEOUT-01' was blocked because dependency-direction strict reported '358' violations and '38' warnings.

The risk is accidental promotion: a generated report, old chat, or polished book can sound authoritative even when it is only evidence.

For current authority and historical context, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around authority order, archive status, non-promotion points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use current authority and historical context as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for authority order, archive status, non-promotion, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. Inconsistent clients, shard drift, and audit failure. Constitutional conflicts are resolved by this document, not by prompt convenience. This is the full canonical glossary v1.0.0. If terminology conflicts with local or legacy docs, this glossary wins.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Speculative future feature not yet implemented. Continue documentation and evidence updates that keep remaining warnings and full-gate debt visible. 'archive/': historical, superseded, quarantined, or provenance-retained. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the

implied implementation step. The following were not finished or intentionally deferred. Use negative architecture rules to prevent drift. Source rules: labels used: "FACT", "INFERENCE", "UNCERTAIN / UNVERIFIED", "PROJECT-CONTEXT" conflict rules. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent.

This book therefore uses historical material as context and asks future tasks to promote only through explicit scoped review.

Chapter 3

Part II - The Architecture

3.1 5. The System Stack

The system stack is a set of ownership boundaries, not just a directory tree.

Current docs distinguish deterministic substrate, official domain layer, product shells, runtime services, contracts, content, tools, and documentation.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. Runtime behavior MUST resolve from profile data ('ExperienceProfile', 'LawProfile', 'ParameterBundle', optional 'MissionSpec' constraints). All runtime-impacting schemas MUST declare 'schema id', 'schema version', and 'stability'.

A second current-thread is the project's insistence that roles remain explicit. Permission to execute Processes under Law. Authority is not power; it is validated capability within constraints. Dominion is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout.

Current documentation also narrows the interpretation of the theme. When behavior or compatibility meaning changes, update the relevant contract surface and proof in the same task. Packs provide capabilities, data, and assets; executables do not embed content. Current repo truth: Dominion is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. Domino is the reusable deterministic substrate: execution ordering, storage, replay, ABI-facing boundaries, and engine mechanisms. Archived conversations consistently treat Domino as reusable infrastructure rather than a one-off game binary.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Domino is the deterministic substrate. Dominion is the official game, product, and domain layer. Workbench is a governed validation and inspection environment. Setup and launcher compose product instances. Tools validate, generate, audit, and migrate. Dominion is not a monolithic game executable, a renderer-owned simulation, a traditional mode-flag game, a silent fallback system, or a place where generated output becomes source truth by convenience. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. The visible

conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority.

Historical material often describes the same system in more speculative or product-facing language, especially around Workbench, providers, and large-scale world simulation.

The conversation corpus adds more texture. Current repo truth: Dominion is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. Domino is the reusable deterministic substrate: execution ordering, storage, replay, ABI-facing boundaries, and engine mechanisms. Archived conversations consistently treat Domino as reusable infrastructure rather than a one-off game binary. Domino is the deterministic substrate. Dominion is the official game, product, and domain layer. Workbench is a governed validation and inspection environment. Setup and launcher compose product instances. Tools validate, generate, audit, and migrate. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The visible conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority. The conversation implies a goal of turning Dominion into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The central architectural conclusion is that Domino should be a framework: it defines stable contracts, service APIs, provider manifests, profiles, capability/refusal law, conformance tests, and deterministic simulation boundaries. A Domino engine. The answer challenged the framing. It said that UE6 should not be treated as a present production target because public UE6 tooling and detailed requirements were not established in the visible sources. It recommended building in current Unreal. The user's underlying goal was to build Dominion as more than a single game executable. The desired result is an extensible ecosystem with a reusable engine, a launcher, setup tools, editors, mod/pack SDKs, game creation tools, and a game that can. The provider strategy became service-first. Raylib, rlg, rls, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominion owns service contracts, provider law, commands, saves, replays, packs, UI documents, and.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is. The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools map to which final homes, which APIs become stable, which code is reusable. The conversation began from an earlier baseline: CLI should always work, TUI should be expected, and multiple native or rendered GUI shells should be thin, replaceable frontends over a stable backend/UI contract. Windows, macOS, Linux, and Android lanes. The conversation quickly shifted from simple prompt engineering toward harness engineering. The central lesson became that high-quality agentic development depends less on one perfect prompt and more on the surrounding system: repo instructions, task.

Several decisions are folded into this topic rather than standing apart from it. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. The visible conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it

enables multiplayer authority. The conversation implies a goal of turning Dominion into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels.

Other decisions remain tentative because the source material records review posture rather than final promotion. The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. Which ideas are final decisions and which are recommendations?

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. The most important conclusion is that determinism is central. Systems are repeatedly specified to avoid hidden randomness, OS timers, platform dependence, uncontrolled floating point, and runtime allocation during simulation ticks. The visible rationale. No src-domain mapping lock, approved layout, repository structure decision, runtime code path, or semantic contract was changed. Profiling MUST NOT use wall-clock time for any authoritative decision.

The governing constraint is practical rather than cosmetic. Client and renderer project data and issue intents; they do not author authoritative outcomes. Do not perform silent migrations or compatibility reinterpretation. 'platform' (39 conversations): runtime shell, platform adapter, renderer boundary, operating-system support, and portability. Sources: Dominion Advanced Simulation and Infrastructure. Do not infer ownership from similarly named roots.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. The rationale is similar: portability and compatibility. Dominion can be more expressive than the C engine, but it still must not depend on modern C++ features unless the constraint changes. The user's underlying goal was not just "make a launcher." The goal was to prevent the launcher from becoming an architectural dumping ground. The launcher needed to be modular, extensible, native-feeling where appropriate, shared across platforms where. This produced the AppShell direction: all products should boot through a unified app shell, select UI mode deterministically, expose commands consistently, and never bypass validation, compatibility checks, or logging.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. Preserve robotic seed-civilisation, science-bounded procedural generation, field layers, limited mothership, robot spawn labs, nanobot blueprinting, no worker NPC labour, machine process graphs, sparse deterministic simulation, and the first-slice plan. The actual repo was not inspected. The exact accepted directory structure, API policy, DDAP profile, compatibility promises, and first pilot module remain unresolved. Main unresolved issues: DDAP acceptance, actual repo structure, stable boundary list, toolchain strictness, first conformance pilot, compatibility guarantees.

The danger is collapsing roots because they look adjacent: engine, game, runtime, apps, packs, schemas, and docs have different kinds of authority.

For the system stack, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around stack ownership, architecture boundaries, physical roots points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use the system stack as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for stack ownership, architecture boundaries, physical roots, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The exact first playable slice remains unresolved. The first provider wedge is clear, but exact sequence between client/workbench gameplay, Robot OS, and gameplay mechanics still needs a formal milestone. The exact Lua version, Linux baseline, current. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same. Early in the conversation, some ideas were explored that later became obsolete. For example, there was an early suggestion that Visual Studio and Xcode projects could be hand-authored as canonical shells. That was later superseded by the user's applied. 7. The current code is promising but not yet the full vision. It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired. DDAP/DIL is proposed, not yet user-ratified.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. 11. Open Questions and Unresolved Issues. This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust. Old repo snapshot/project attachments, not yet verified.

The downstream effect is that future work has to account for more than the immediate feature request. A 14-prompt Codex implementation roadmap exists.

The useful outcome is a stack that can grow without turning every future feature into a rewrite of the whole project.

3.2 6. Domino, Dominion, Runtime, Products, and Tools

Domino and Dominion are related but not interchangeable.

Domino is the reusable deterministic substrate; Dominium is the official product and domain layer that gives the substrate meaning.

The current material supports that picture in several ways. Dominium is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The conversation implies a goal of turning Dominium into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later.

A second current-thread is the project's insistence that roles remain explicit. What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominium owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools map to which final homes, which APIs become stable, which code is reusable. Inspect the repository, then run or adapt CODEX PROMPT 1. Do not execute later prompts until Prompt 1 establishes specs, invariants, and scaffolding or until repository reality is mapped.

Current documentation also narrows the interpretation of the theme. All mutation must happen through Processes. The conversation then expanded from one subsystem-resources, items, recipes, entities, and extraction mechanics-into almost the entire engine and product ecosystem. We discussed how resource fields should work, how players should build factories, how. Build tools that can help create setup, launcher, game, and future tools. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. How are 'DOMINIUM RUN ROOT' and 'DOMINIUM HOME' finalized? Verify actual repository state, then execute or audit the generated "Dominium Setup: Final Spec Alignment, Gap Closure, and Hardening Pass" prompt.

The conversations add pressure toward many surfaces: client, server, launcher, setup, Workbench, command views, and automation.

The conversation corpus adds more texture. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The conversation implies a goal of turning Dominium into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later. What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.?

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominium owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools

map to which final homes, which APIs become stable, which code is reusable. Inspect the repository, then run or adapt CODEX PROMPT 1. Do not execute later prompts until Prompt 1 establishes specs, invariants, and scaffolding or until repository reality is mapped. All mutation must happen through Processes.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The conversation then expanded from one subsystem-resources, items, recipes, entities, and extraction mechanics-into almost the entire engine and product ecosystem. We discussed how resource fields should work, how players should build factories, how. Build tools that can help create setup, launcher, game, and future tools. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. How are 'DOMINIUM RUN ROOT' and 'DOMINIUM HOME' finalized? Verify actual repository state, then execute or audit the generated "Dominium Setup: Final Spec Alignment, Gap Closure, and Hardening Pass" prompt. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and.

Several decisions are folded into this topic rather than standing apart from it. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The conversation implies a goal of turning Dominium into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation.

Other decisions remain tentative because the source material records review posture rather than final promotion. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. How are 'DOMINIUM RUN ROOT' and 'DOMINIUM HOME' finalized? Verify actual repository state, then execute or audit the generated "Dominium Setup: Final Spec Alignment, Gap Closure, and Hardening Pass" prompt.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Renderer selection follows the standard policy: explicit selection fails loudly when unavailable, and auto selection logs the chosen backend. The descriptor does not own the implementation of those services. Runtime services remain governed by service/provider law. Pack activation remains governed by package and trust law. Module enablement remains governed by module law and the composition. Date: 2026-03-05 Scope: CHEM-2 phases 8-11 completion (proof/replay, enforcement, TestX, final gate report). No src-domain mapping lock, approved layout, repository structure decision, runtime code path, or semantic contract was changed.

The governing constraint is practical rather than cosmetic. 'platform' (39 conversations): runtime shell, platform adapter, renderer boundary, operating-system support, and portability. Sources: Dominium Advanced Simulation and Infrastructure. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. Check live repo state and determine whether QUEUE-RECONCILE-01, REPLAY-PROOF-SLICE-01, and BAREBONES-CLIENT-SHELL-01 have run. Then generate the next missing prompt. Verify every implementation, platform, tooling, release, and queue

claim against current repo artifacts.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. The preserved rule should be: do not make worker NPCs the default mechanism for construction, production, or city simulation. Use automation, AI agents, machine controllers, drones, construction swarms, logistics graphs, and process abstractions. A key process doctrine emerged: development is non-blocking, promotion is evidence-blocked. AIDE should manage parallel task branches, dirty worktree classification, blockers, repair tasks, checkpoints, and main promotion. Full CTest remains full-gate. The central problem was not "how do we open a game window?" It was how to build a long-lived ecosystem around the game without corrupting the deterministic engine or creating hidden coupling. The launcher needed to be powerful enough to manage multiple. This produced the AppShell direction: all products should boot through a unified app shell, select UI mode deterministically, expose commands consistently, and never bypass validation, compatibility checks, or logging.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. The most important unresolved issue is actual repository state. We do not know which prompts were run, which files exist, or which systems are implemented. Do not assume unresolved items are safe to implement. This prompt is important because it bridges the gap between architectural planning and implementation cleanup. It says: do not redesign setup; finish, document, and align what exists. UI is data (UI IR), never business logic.

That pressure is useful only if products remain projections and operators over the substrate instead of becoming alternate authorities.

For domino, dominium, runtime, products, and tools, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around Domino/Dominium split, product shells, operator tools points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use domino, dominium, runtime, products, and tools as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for Domino/Dominium split, product shells, operator tools, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Workbench remains unresolved at the implementation level: read-only first is agreed, but exact UI shell design, renderer path, modules, and self-hosting milestones remain future tasks. Early in the conversation, some ideas were explored that later became obsolete. For example, there was an early suggestion that Visual Studio and Xcode projects could be hand-authored as canonical shells. That was later superseded by the user's applied. The unique contribution is the synthesis: Dominium should not be framed only as a game or editor suite; it should

be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. Codex prompts were generated; application status is unknown.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust. Old repo snapshot/project attachments, not yet verified. Unresolved goals include confirming live repo state, executing pending maintenance and projection prompts, building read-only Workbench, implementing provider manifests/fences, building raylib provider wedge, proving Universe Explorer. VM-only vs native plugin policy is unresolved.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. What are the known warnings and blockers? 'PACKAGE-MOUNT-SLICE-01' was reported as complete with 'PASS WITH WARNINGS', and the next intended product-spine task was 'REPLAY-PROOF-SLICE-01'. Reason: The old plan risked becoming a separate one-off tool architecture. Workbench can instead prove and reuse the same systems as the client, launcher, setup, server, and future tools. The old "UI Editor / Tool Editor" idea was superseded. The preferred model is Dominion Workbench as the production environment for building the project.

The downstream effect is that future work has to account for more than the immediate feature request. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation. Affects: UI structure, module order, implementation prompts. The discussion then mapped these ideas onto Dominion's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX, BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We.

A later task touching this area should therefore state its validation path before it starts changing behavior. A 14-prompt Codex implementation roadmap exists.

The project becomes easier to evolve when runtime mechanisms, product shells, and repo tools each keep their role.

3.3 7. Law, Capability, Refusal, and Evidence

Law and capability are not decoration; they are how the system decides what may happen.

Current authority requires explicit refusal, no hidden fallback behavior, and capability-driven access to optional content and operations.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. Travel scheduling, replay, and law decisions become nondeterministic. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly.

A second current-thread is the project's insistence that roles remain explicit. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Reachability lies, bypassed costs, and broken domain law. Invariant: No authority without capability and law. Dominion/Domino is a deterministic world runtime and law-governed simulation OS.

Current documentation also narrows the interpretation of the theme. A multi-scale reality model with explicit existence/refinement and visitability. There are no game modes; behavior is law + capability + policy. Au-

thoritative logic is deterministic and law-gated. There are no hard-coded modes; laws and capabilities define behavior.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Admin power is capability- and law-gated, audited, and fork-aware. It supports macro-only, meso, and micro refinement with explicit contracts. Replays diverge, cross-shard reconciliation fails, and saves become invalid. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints.

Archive material shows the same pattern being applied to providers, packs, assistant workflows, Workbench commands, and future product surfaces.

The conversation corpus adds more texture. Any claim touching canon, glossary, authority order, contracts, schema law, current queue, release, implementation, or blocked scope. The conversations strongly prefer a rich Workbench and eventual editor/operator surfaces. Current authority is narrower: broad Workbench UI, renderer implementation, and native GUI remain blocked. Therefore synthesis may describe intent, but follow-up. Do not patch canon, contracts, schema, implementation, release, or queue files from this atlas. The conversation established that *Dominium* should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The central architectural conclusion is that *Domino* should be a framework: it defines stable contracts, service APIs, provider manifests, profiles, capability/refusal law, conformance tests, and deterministic simulation boundaries. A *Domino* engine. C17 + C++17 mainline; C-compatible ABI; deterministic fixed-width law; 64-bit source-native; little-endian; providers not identity; Workbench not authority; Foundation blocker first. The user's underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of *Dominium*: a deterministic, moddable, inspectable, replayable universe platform where client, server. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Reader Brief - *Dominium* Canon, Repository Alignment, and Portability Doctrine. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work.

The archive also records how the project repeatedly returned to the same pressure points from different directions. SELF-AUDIT AND REVISION - *Dominium* Canon, Repository Alignment, and Portability Doctrine. Separated repo docs, schemas, code, and tests in maturity claims. Reliability rating: 4/5 with explicit caveats. How should public API/ABI boundaries be written?

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. Support canonical surface plus multiple surface instances for shards/mirrors.

Other decisions remain tentative because the source material records review posture rather than final promotion. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. *Domino* should store the true universe as seeds, rules, fields, sparse. The most important repo-supported decision is that *Dominium*

authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Domain permissions + existence/refinement - visitability decision. Which decisions are confirmed versus recommendations? Review decisions with uncertainty labels, then formalise the vertical slice.

The governing constraint is practical rather than cosmetic. Do not perform silent migrations or compatibility reinterpretation. Save/replay integrity depends on identical macro transitions. Scope: single-source dependency map and forbidden edges for Dominion/Domino. Scope: canonical statement of identity and intent.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. "Convenient" implicit behavior is not allowed. Game code depends on logical components, not layout or memory order. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days.

The hard part is refusing clearly without making the system brittle or opaque.

For law, capability, refusal, and evidence, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around law surfaces, capability checks, refusal discipline points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use law, capability, refusal, and evidence as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for law surfaces, capability checks, refusal discipline, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. Inconsistent clients, shard drift, and audit failure. Crosses unresolved or quarantined ownership areas. 'archive/': historical, superseded, quarantined, or

provenance-retained.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Use negative architecture rules to prevent drift. The main managed risks were drift, over-abstractation, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. Another tradeoff is between a clean theoretical structure and the current repo's transitional debt. The current ownership-root target is good, but the repo still has cleanup work. The safe path is not broad movement by directory name; it is gated. Included specific risks about overclaiming implementation status.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

A well-formed future feature should be able to explain its authority source, its refusal path, and the evidence it leaves behind.

3.4 8. Determinism, Replay, and Provenance

Determinism is both a runtime requirement and an audit strategy.

The current doctrine asks for named randomness, deterministic ordering, stable reductions, replay equivalence, and provenance wherever canonical truth is involved.

The current material supports that picture in several ways. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. Domino is the reusable deterministic substrate: execution ordering, storage, replay, ABI-facing boundaries, and engine mechanisms. Archived conversations consistently treat Domino as reusable infrastructure rather than a one-off game binary. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts.

A second current-thread is the project's insistence that roles remain explicit. The conversation established that Dominium should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts. The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominium owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is. The conversation quickly shifted from simple prompt engineering toward harness engineering. The central lesson became that high-quality agentic development depends less on one perfect prompt and more on the surrounding system: repo instructions, task.

Current documentation also narrows the interpretation of the theme. Net handshake includes build ID, schema ID/version, determinism profile, content hashes. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. Reader Brief - Dominion Modularity, AIDE Refactorability, and Future-Proof Architecture.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Verify live repo head, CTest/validator status, referenced docs, old tool paths, component matrix names, and prior generated file availability. Domino/Dominium future-proof architecture: standards-informed assurance, portability, modularity, extensibility, replaceability, public contracts, data compatibility, deterministic replay, testing, and preservation. Most important unresolved issue: Domino's exact role and capabilities are not defined in the visible chat. The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools map to which final homes, which APIs become stable, which code is reusable.

Conversation evidence adds operational concerns: proof gates, CI tiers, source evidence, dependency warnings, portability, and release trust.

The conversation corpus adds more texture. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. Domino is the reusable deterministic substrate: execution ordering, storage, replay, ABI-facing boundaries, and engine mechanisms. Archived conversations consistently treat Domino as reusable infrastructure rather than a one-off game binary. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. The conversation established that Dominion should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominion owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is. The conversation quickly shifted from simple prompt engineering toward harness engineering. The central lesson became that high-quality agentic development depends less on one perfect prompt and more on the surrounding system: repo instructions, task. Net handshake includes build ID, schema ID/version, determinism profile, content hashes.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. Reader Brief - Dominion Modularity, AIDE Refactorability, and Future-Proof Architecture. Verify live repo head, CTest/validator status, referenced docs, old tool paths, component matrix names, and prior generated file availability.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Domino/Dominium future-proof architecture: standards-informed assurance, portability, modularity, extensibility, replaceability, public contracts, data compatibility, deterministic replay, testing, and preservation. Most important unresolved issue: Domino's exact role and capabilities are not defined in the visible chat. The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools map to which final homes, which APIs become stable, which code is reusable. The conversation consistently preserved this boundary. The engine should define

deterministic mechanisms; the game defines rules and meaning. This affects directory layout, APIs, naming, dependencies, modding, and future reuse. It would be revisited only.

Several decisions are folded into this topic rather than standing apart from it. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. The fast tier found and fixed an AIDE self-test writer compatibility issue before final validation.

Other decisions remain tentative because the source material records review posture rather than final promotion. Logs must record control decisions and reasons. Date: 2026-03-05 Scope: CHEM-4 phases 0-9 completion (stress envelope, deterministic degradation, conservation verification, proof/replay, regression lock, enforcement, TestX, final baseline). Date: 2026-03-05 Scope: CHEM-3 phases 11-12 completion (TestX coverage, gate validation, final baseline report). Date: 2026-03-05 Scope: CHEM-2 phases 8-11 completion (proof/replay, enforcement, TestX, final gate report).

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. 'session begin' binds the connection to the chosen contract bundle hash and pack lock hash. Provenance record: identifiers for inputs, models, and decisions.

The governing constraint is practical rather than cosmetic. Do not perform silent migrations or compatibility reinterpretation. Topic 8 - Determinism, data, compatibility, and testing. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. Series Scope: repo-structure discovery and design Series Role: authoritative preferred-target and boundary packet for later migration, shim, rollback, and ownership-reconciliation prompts; downstream of stronger canon, the Omega0 constraint packet, the.

Those constraints matter because they prevent future work from using the book as an implementation short-cut. Series Scope: repo-structure discovery and design Series Role: risk and coupling packet for later topology-option, preferred-target, shim-design, migration-sequencing, and ownership-reconciliation prompts; downstream of stronger canon, the Omega0. Series Scope: repo-structure discovery and design Series Role: authoritative target-topology option set for later preferred-target selection, shim-design, and migration-sequencing prompts; downstream of stronger canon, the Omega0 constraint packet, the. Series Scope: repo-structure discovery and design Series Role: authoritative topology, path-contract, and ownership reality map for later repo-structure discovery, design, migration-planning, and decision prompts; downstream of stronger canon, the Omega0. Signals flow through explicit interfaces. Compatibility checks are deterministic and based on declared type, directionality, capacity, and impedance class. Partial compatibility is expressed as degradation (noise/latency envelopes), never implicit.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. RepoX cold-path structural invalidation was narrowed to scoped groups with artifact-aware dep hashing and per-group cache reuse. The local Python 3.8.1 'Path.write text' implementation does not support the 'newline' keyword. AIDE Lite already normalizes text before writing, so the safe compatibility fix is to open the file explicitly with 'newline="\n"'. Worktree after: scoped closeout evidence/status changes pending commit. Changelog previews are deterministic local evidence. They may be generated for review, but they do not publish releases, create tags, upload assets, or mutate remote repository state.

A feature that cannot describe its replay and evidence behavior is not ready to become authoritative.

For determinism, replay, and provenance, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around determinism, replay, proof, provenance points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior,

and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use determinism, replay, and provenance as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for determinism, replay, proof, provenance, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Tiny sequential micro-move waves were superseded by deterministic routing and later by governed parallel task work.

The downstream effect is that future work has to account for more than the immediate feature request. The 'src' issue illustrates the reasoning. A generic 'src' root is not bad in all projects, but in *Dominium* it hides ownership and allowed Codex/humans to create shadow implementations. Therefore, the policy became ownership-rooted layout rather than.

The design consequence is that validation is not an afterthought; it is part of the architecture.

3.5 9. Contracts, Schemas, Packs, and Compatibility

Contracts and schemas are how public meaning survives change.

Current docs require explicit identities, versions, migrations, refusal obligations, and compatibility rules.

The current material supports that picture in several ways. Foundation Lock means the governance spine exists, validates in required scope, is documented, and is integrated with the normal fast strict proof gate. It allows only narrow product slices that obey the public surface, command, diagnostics, artifact. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout. Packs provide capabilities, data, and assets; executables do not embed content.

A second current-thread is the project's insistence that roles remain explicit. This authorization remains narrow. It does not authorize broad Workbench UI, gameplay, renderer, native GUI, runtime provider expansion, package runtime, release publication, or domain feature work. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. Final active plan is the 9-prompt pack, not earlier superseded plans. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt.

Current documentation also narrows the interpretation of the theme. Write the implementation prompt for public surface registry. Main open questions: dependency versions, C17/C++17 repo state, Lua version, framework ABI, profile order, deterministic class, license policy, CAD graph scope, and distributed compute verification.

C17 + C++17 mainline; C-compatible ABI; deterministic fixed-width law; 64-bit source-native; little-endian; providers not identity; Workbench not authority; Foundation blocker first. See Task Register in file 04. Highest priority: Release Constitution, SemVer Component Inventory, suite version policy, GBN/BII spec, capability registry.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Exact suite semantics, exact SemVer component list, BII schema, filename grammar, manifest schema, target taxonomy, and capability registry. A major later phase shifted from pure architecture into repo-specific planning. The repo had undergone extensive CONVERGE and POST-CONVERGE work: layout contracts, root allowlists, stale doc supersession, root recycling, AIDE control plane, build proof. The user's underlying goal was to build Dominium as more than a single game executable. The desired result is an extensible ecosystem with a reusable engine, a launcher, setup tools, editors, mod/pack SDKs, game creation tools, and a game that can. The exact first implementation branch, versions, and dependency pins are unresolved. The exact Domino Framework ABI is unresolved. The exact Lua version is unresolved. The current repository baseline must be verified. The formal policy for.

The archive adds a larger vocabulary around packs, providers, module boundaries, package composition, semantic versioning, and release identity.

The conversation corpus adds more texture. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. Final active plan is the 9-prompt pack, not earlier superseded plans. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Write the implementation prompt for public surface registry. Main open questions: dependency versions, C17/C++17 repo state, Lua version, framework ABI, profile order, deterministic class, license policy, CAD graph scope, and distributed compute verification. C17 + C++17 mainline; C-compatible ABI; deterministic fixed-width law; 64-bit source-native; little-endian; providers not identity; Workbench not authority; Foundation blocker first. See Task Register in file 04. Highest priority: Release Constitution, SemVer Component Inventory, suite version policy, GBN/BII spec, capability registry.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Exact suite semantics, exact SemVer component list, BII schema, filename grammar, manifest schema, target taxonomy, and capability registry. A major later phase shifted from pure architecture into repo-specific planning. The repo had undergone extensive CONVERGE and POST-CONVERGE work: layout contracts, root allowlists, stale doc supersession, root recycling, AIDE control plane, build proof. The user's underlying goal was to build Dominium as more than a single game executable. The desired result is an extensible ecosystem with a reusable engine, a launcher, setup tools, editors, mod/pack SDKs, game creation tools, and a game that can. The exact first implementation branch, versions, and dependency pins are unresolved. The exact Domino Framework ABI is unresolved. The exact Lua version is unresolved. The current repository baseline must be verified. The formal policy for.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The conversation began from an earlier baseline: CLI should always work, TUI should be expected, and multiple native or rendered GUI shells should be thin, replaceable frontends over a stable backend/UI contract. Windows, macOS, Linux, and Android lanes. Net handshake includes build ID, schema ID/version, determinism profile, content hashes. The user's underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of Dominium: a deterministic, moddable, inspectable, replayable universe platform where client, server. Top tasks: verify repo baseline, draft framework ABI, add provider manifests, add forbidden include validator, implement null providers, implement raylib providers, create profiles, and define work-lease/CAD schemas.

Several decisions are folded into this topic rather than standing apart from it. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. Final active plan is the 9-prompt pack, not earlier superseded plans. Engine, game, contracts, content, docs, tests, tools, and release roots were routed toward the final ownership grammar.

Other decisions remain tentative because the source material records review posture rather than final promotion. Chosen provider capabilities flow into CAP-NEG and may drive explicit degrade/refuse outcomes. 'session begin' binds the connection to the chosen contract bundle hash and pack lock hash.

The governing constraint is practical rather than cosmetic. 'platform' (39 conversations): runtime shell, platform adapter, renderer boundary, operating-system support, and portability. Sources: Dominion Advanced Simulation and Infrastructure. Mass moves or broad rewrites can break build, tests, imports, packaging, and compatibility. Use small gated migrations. The user's underlying goal was not just "make a launcher." The goal was to prevent the launcher from becoming an architectural dumping ground. The launcher needed to be modular, extensible, native-feeling where appropriate, shared across platforms where. The immediate plan is not more doctrine. The next active task is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01 . It should classify the 358 dependency-direction violations and 38 warnings, repair real boundary leaks, add only narrow justified exceptions.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. The actual repo was not inspected. The exact accepted directory structure, API policy, DDP profile, compatibility promises, and first pilot module remain unresolved. 2. One GUI per OS version. Rejected as matrix explosion. The better model is compatibility lanes and visual profiles. VERIFY-11: Android TV, Google TV, Android Automotive, Chromebook Android compatibility, and vendor ROM constraints. The mothership should also explain why players have recipes and do not need to invent CPUs, rail gauges, calendars, or basic industrial theory from scratch. The ship already carries human knowledge. The progression challenge is not discovering the idea.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. The proposed long-term policy is that released IDs are never reused and old data gets fixtures and migration tests. Do not assume unresolved items are safe to implement. The project includes planets, civilisations, terraforming, factories, megaprojects, and MMO persistence. The immediate risk is trying to build all of it before proving the smallest fun loop. WinForms old .NET support needs real target/build lanes; verify before use.

Some of that vocabulary is current contract law, while some remains proposal or future queue work.

For contracts, schemas, packs, and compatibility, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around contracts, schemas, packs, compatibility points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use contracts, schemas, packs, and compatibility as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for contracts, schemas, packs, compatibility, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate. The exact first playable slice remains unresolved. The first provider wedge is clear, but exact sequence between client/workbench gameplay, Robot OS, and gameplay mechanics still needs a formal milestone. The exact Lua version, Linux baseline, current. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The conversation did not yet define how those product backends are exposed. That remains the most important unresolved design issue.

The reader should separate data-driven extension that is already doctrinal from provider or package runtime ambitions that remain blocked.

Chapter 4

Part III - Product and Operator Surfaces

4.1 10. Client, Server, Launcher, Setup, and Workbench

The product surfaces are where users and operators meet the governed substrate.

Current repo state permits narrow product and validation slices, not broad product implementation.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Dominion is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. Domino is the deterministic substrate. Dominion is the official game, product, and domain layer. Workbench is a governed validation and inspection environment. Setup and launcher compose product instances. Tools validate, generate, audit, and migrate. The archive describes Dominion as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not.

A second current-thread is the project's insistence that roles remain explicit. This authorization remains narrow. It does not authorize broad Workbench UI, gameplay, renderer, native GUI, runtime provider expansion, package runtime, release publication, or domain feature work. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly.

Current documentation also narrows the interpretation of the theme. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Dominion/Domino is a deterministic world runtime and law-governed simulation OS. There are no game modes; behavior is law + capability + policy. What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.?

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Dominion is not a monolithic game executable, a renderer-owned simulation, a traditional mode-flag game, a silent fallback system, or a place where generated output becomes source truth by convenience. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into

a fabricated robot body, and disconnects into. The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominion owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is.

The archive contains rich Workbench and launcher ambition: inspection, commands, evidence, authoring, package views, and later editing workflows.

The conversation corpus adds more texture. Domino is the deterministic substrate. Dominion is the official game, product, and domain layer. Workbench is a governed validation and inspection environment. Setup and launcher compose product instances. Tools validate, generate, audit, and migrate. The archive describes Dominion as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not. ”: What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominion owns service contracts, provider law, commands, saves, replays, packs, UI documents, and. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The user’s underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of Dominion: a deterministic, moddable, inspectable, replayable universe platform where client, server. Build tools that can help create setup, launcher, game, and future tools. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. Client compute must not commit persistent MMO outcomes. Which parts should become spec requirements?

Several decisions are folded into this topic rather than standing apart from it. ”: What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. The conclusion was not to globally simulate everything. Instead, the architecture should

support deterministic lazy historical evaluation.

Other decisions remain tentative because the source material records review posture rather than final promotion. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. This pass continued canonical structure convergence after CANON STRUCTURE FINALIZE NOW 01. It was limited to structure, naming, ownership, validator, generated-map, and proof cleanup. It did not implement runtime, product, Workbench, renderer, native. Renderer selection follows the standard policy: explicit selection fails loudly when unavailable, and auto selection logs the chosen backend. 3. Replace control-plane inline downgrade logic with kernel invocation. 4. Replace MAT-9 fidelity fallback with kernel negotiation result (no inline downgrade policy). 5. Replace MAT-7 budget truncate decision path with kernel arbitration output (engine. No src-domain mapping lock, approved layout, repository structure decision, runtime code path, or semantic contract was changed.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Game code depends on logical components, not layout or memory order. Check live repo state and determine whether QUEUE-RECONCILE-01, REPLAY-PROOF-SLICE-01, and BAREBONES-CLIENT-SHELL-01 have run. Then generate the next missing prompt. The repo is much healthier. The broad directory crisis appears to be over. The structure is not perfect, and the proof pipeline is not full green. But the project now has a credible canonical root model, structure validators, provider structure doctrine.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. A key process doctrine emerged: development is non-blocking, promotion is evidence-blocked. AIDE should manage parallel task branches, dirty worktree classification, blockers, repair tasks, checkpoints, and main promotion. Full CTest remains full-gate. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. The immediate plan is not more doctrine. The next active task is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01 . It should classify the 358 dependency-direction violations and 38 warnings, repair real boundary leaks, add only narrow justified exceptions. This produced the AppShell direction: all products should boot through a unified app shell, select UI mode deterministically, expose commands consistently, and never bypass validation, compatibility checks, or logging.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. The preserved rule should be: do not make worker NPCs the default mechanism for construction, production, or city simulation. Use automation, AI agents, machine controllers, drones, construction swarms, logistics graphs, and process abstractions. If you want to continue this work, do not restart with UI Editor. Start from Workbench and the presentation spine. The live repo check showed Foundation Lock initially blocked by dependency-direction strict validation. This was the first serious foundation validator still red: 358 violations and 38 warnings. The next prompt was generated to repair dependency. Do not start broad Workbench UI, renderer, native GUI, package runtime, provider runtime, module loader, gameplay, or public release yet.

Those ideas matter because they show the desired operator experience, but the queue still controls when they can become implementation.

For client, server, launcher, setup, and workbench, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read

together. The recurring material around client/server/product shells, Workbench, launcher/setup points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use client, server, launcher, setup, and workbench as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for client/server/product shells, Workbench, launcher/setup, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Inconsistent clients, shard drift, and audit failure. The exact first playable slice remains unresolved. The first provider wedge is clear, but exact sequence between client/workbench gameplay, Robot OS, and gameplay mechanics still needs a formal milestone. The exact Lua version, Linux baseline, current. Workbench remains unresolved at the implementation level: read-only first is agreed, but exact UI shell design, renderer path, modules, and self-hosting milestones remain future tasks. Reason: The old plan risked becoming a separate one-off tool architecture. Workbench can instead prove and reuse the same systems as the client, launcher, setup, server, and future tools.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. Old repo snapshot/project attachments, not yet verified. Unresolved goals include confirming live repo state, executing pending maintenance and projection prompts, building read-only Workbench, implementing provider manifests/fences, building raylib provider wedge, proving Universe Explorer. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. 12. Risks, Failure Modes, and What Future Chats Might Get Wrong. What are the known warnings and blockers? 'PACKAGE-MOUNT-SLICE-01' was reported as complete with 'PASS WITH WARNINGS', and the next intended product-spine task was 'REPLAY-PROOF-SLICE-01'. What superseded the old UI Editor plan?

The downstream effect is that future work has to account for more than the immediate feature request. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

Workbench should be read as the richest future operator surface, not as an authority layer.

4.2 11. UI, Renderer, Native GUI, and Presentation Boundaries

Presentation is downstream of truth.

Current doctrine says rendering is presentation only, and current queue state keeps broad renderer, native GUI, and UI implementation blocked.

The current material supports that picture in several ways. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Dominium is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based.

A second current-thread is the project's insistence that roles remain explicit. Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003). Micro detail is refined only when required; macro existence is valid. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR. Dominium is not a monolithic game executable, a renderer-owned simulation, a traditional mode-flag game, a silent fallback system, or a place where generated output becomes source truth by convenience.

Current documentation also narrows the interpretation of the theme. It is built to host multi-scale universes with explicit existence, travel,. It is not a generic rendering or physics framework; rendering is derived and. Verify latest queue. If unchanged, proceed with 'PRESENTATION-CONTRACT-01'; then draft 'UNIVERSE-EXPLORER-CONTRACT-01'. The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. Most important: actual repo structure, existing GPT-5.2 refactor plan contents, final Q formats, orientation representation, grid assumptions in current code, microsegment length, packing VM instruction set, bridge/tunnel ownership, and checksum rules. Read file 01 first if you want to understand what happened. Read file 04 if you need tasks and decisions. Read file 05 if merging into a master Project Spec Book. Read file 07 before treating external support claims as implementation truth. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and.

The conversation archive nevertheless contains extensive UI and renderer intent, including universe exploration, native surfaces, and product presentation strategy.

The conversation corpus adds more texture. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR. Verify latest queue. If unchanged, proceed with 'PRESENTATION-CONTRACT-01'; then draft 'UNIVERSE-EXPLORER-CONTRACT-01'. The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. A

central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. Most important: actual repo structure, existing GPT-5.2 refactor plan contents, final Q formats, orientation representation, grid assumptions in current code, microsegment length, packing VM instruction set, bridge/tunnel ownership, and checksum rules. Read file 01 first if you want to understand what happened. Read file 04 if you need tasks and decisions. Read file 05 if merging into a master Project Spec Book. Read file 07 before treating external support claims as implementation truth. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Actual repo, build system, dsys/dgfx APIs were not inspected. Which parts should become spec requirements? The user's underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of Dominion: a deterministic, moddable, inspectable, replayable universe platform where client, server. Calendars are pure renderers and never affect physics.

The archive also records how the project repeatedly returned to the same pressure points from different directions. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Write the implementation prompt for public surface registry.

Several decisions are folded into this topic rather than standing apart from it. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. A central decision was to separate the project into two conceptual layers. Domino is the reusable engine layer: platform abstraction, renderer abstraction, audio, UI, core services, package/instance management, simulation control, events, plugins. Most important: actual repo structure, existing GPT-5.2 refactor plan contents, final Q formats, orientation representation, grid assumptions in current code, microsegment length, packing VM instruction set, bridge/tunnel ownership, and checksum rules.

Other decisions remain tentative because the source material records review posture rather than final promotion. Read file 01 first if you want to understand what happened. Read file 04 if you need tasks and decisions. Read file 05 if merging into a master Project Spec Book. Read file 07 before treating external support claims as implementation truth. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Renderer selection follows the standard policy: explicit selection fails loudly when unavailable, and auto selection logs the chosen backend. Commitments are strongly recommended and required for process families that already define commitment artifacts (construction, logistics, maintenance), but missing commitment does not globally refuse all macro changes. Launcher integration flags are accepted but not required: '--launcher-' (ignored by the game runtime unless explicitly wired).

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Responsibilities and forbidden dependencies (quick map). Verify the latest repo queue and Foundation Lock, then draft 'PRESENTATION-CONTRACT-01' if it is not complete; if it is complete, draft 'UNIVERSE-EXPLORER-

CONTRACT-01' as a planning artifact, preserving Workbench projection discipline. Verify every implementation, platform, tooling, release, and queue claim against current repo artifacts.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. The code audit concluded that the current repo is modular at source/build level and partly data-driven through registries and validators, but not yet a fully data-driven moddable runtime. The client still has hard-coded help, command descriptors, session. A key process doctrine emerged: development is non-blocking, promotion is evidence-blocked. AIDE should manage parallel task branches, dirty worktree classification, blockers, repair tasks, checkpoints, and main promotion. Full CTest remains full-gate. AIDE was positioned as a build intelligence and evidence layer. It should probe the machine, explain which tuples are available or blocked, generate local presets, run tuple verification, and record evidence. XStack should remain umbrella orchestration. If you want to continue this work, do not restart with UI Editor. Start from Workbench and the presentation spine.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. The live repo check showed Foundation Lock initially blocked by dependency-direction strict validation. This was the first serious foundation validator still red: 358 violations and 38 warnings. The next prompt was generated to repair dependency. A live repo check during the conversation found Foundation Lock initially blocked by dependency-direction strict validation. The immediate next action is not more architecture brainstorming. The latest project task/review packets say Foundation Lock is blocked because dependency-direction strict validation reports 358 violations and 38 warnings. The next task is. Do not assume unresolved items are safe to implement.

The book keeps those ideas as design context because they explain the desired experience, but it does not treat them as permission.

For ui, renderer, native gui, and presentation boundaries, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around renderer/UI/native GUI, presentation separation points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use ui, renderer, native gui, and presentation boundaries as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for renderer/UI/native GUI, presentation separation, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Inconsistent clients, shard drift, and audit failure. Refusal and defer semantics are part of determinism and replay guarantees. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. Resolve DSYS vs 'domino sys' if conflict appears.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. The major unresolved issue is taxonomy: how exactly POSIX, SDL1, SDL2, and Native map to Win32, Cocoa, Carbon, X11, Wayland, headless, and retro platforms. The conversation did not yet define how those product backends are exposed. That remains the most important unresolved design issue. Workbench remains unresolved at the implementation level: read-only first is agreed, but exact UI shell design, renderer path, modules, and self-hosting milestones remain future tasks.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. TUI is expected for every product, but parity level is unresolved. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt. 12. Risks, Failure Modes, and What Future Chats Might Get Wrong. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer.

The downstream effect is that future work has to account for more than the immediate feature request. Affects: UI structure, module order, implementation prompts. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

Future renderer work will need explicit queue opening and contracts that protect truth, perception, replay, and refusal.

4.3 12. AIDE, Codex, Automation, and Repo Governance

AIDE, Codex, and related agents are governed helpers, not alternate project authorities.

Agent governance defines bounded work, validation expectations, protected paths, and forbidden moves.

The current material supports that picture in several ways. This document is the binding constitutional contract for Dominion architecture and execution. If any code, schema, tool, prompt, or documentation conflicts with this contract, this contract wins. Terms in this document are canonical and binding for contracts, schemas, code comments, and prompts. Run 'AIDE-WORKFLOW-LAW-01' next if the queue remains reconciled after. Use this prompt block for future work so tasks remain short and stable.

A second current-thread is the project's insistence that roles remain explicit. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints. Dependency direction is authoritative and must not be inverted. Law admission gates sit before scheduling, before execution, and before commit. Scope: hard invariants for all systems, documentation, and data.

Current documentation also narrows the interpretation of the theme. Invariant: Authority gates actions only, never visibility (AUTH3-AUTH-001). Invariant: Authority upgrades/downgrades do not mutate state (AUTH3-UPGRADE-007). Invariant: Saves are tagged by authority scope (AUTH3-SAVE-008). A law-governed simulation OS where authority is data-defined.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. God-mode admin tools (law-gated and audited). It is not acceptable to bypass law or audit for convenience. Authority cannot bypass audit or law gates. Non-use: canon, contract/schema, implementation, release, or queue override.

Conversation evidence shows repeated work to make assistant operation more portable, evidence-rich, and reviewable.

The conversation corpus adds more texture. This atlas is a reader map. It is not canon and does not promote archived conversation claims. This map combines current repo orientation with conversation-derived design intent. When they differ, current repo authority wins. A large set of prompts was then generated to lock down

architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The final plan accepted the newer repo structure is clean enough to stop giant structure prompts. Remaining cleanup should be targeted: stale full-gate tests, pack layout canon, residual taxonomy, AIDE state classification, public header ABI. Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?". Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. The conversation established that Dominion should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. See Task Register in file 04. Highest priority: Release Constitution, SemVer Component Inventory, suite version policy, GBN/BII spec, capability registry. Exact suite semantics, exact SemVer component list, BII schema, filename grammar, manifest schema, target taxonomy, and capability registry. A major later phase shifted from pure architecture into repo-specific planning. The repo had undergone extensive CONVERGE and POST-CONVERGE work: layout contracts, root allowlists, stale doc supersession, root recycling, AIDE control plane, build proof. The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics, evidence, tests, and replay/proof. It is.

The archive also records how the project repeatedly returned to the same pressure points from different directions. The conversation quickly shifted from simple prompt engineering toward harness engineering. The central lesson became that high-quality agentic development depends less on one perfect prompt and more on the surrounding system: repo instructions, task. This reader page is derived from archived conversation material. It cannot override canon, contracts, schema law, current queue state, live repo structure, or validated repo artifacts. The user's central concern was that the project must be built to last. The desired system is not a throwaway indie codebase. It should be portable, modular, extensible, reusable for different games on the same Domino engine, reusable for other engine. That cleanup prompt instructed Codex to remove the redundant block, remove undefined "embedded" from the future systems list, replace vague "lockstep/rollback" wording with lockstep-first canonical networking language, and update the contributing.

Several decisions are folded into this topic rather than standing apart from it. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too. The final plan accepted the newer repo structure is clean enough to stop giant structure prompts. Remaining cleanup should be targeted: stale full-gate tests, pack layout canon, residual taxonomy, AIDE state classification, public header ABI. Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?".

Other decisions remain tentative because the source material records review posture rather than final promotion. Support canonical surface plus multiple surface instances for shards/mirrors. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. The final state of the conversation is therefore: the architecture and implementation-roadmap discussion has been preserved, but the actual next work is elsewhere-especially the Path D advanced simulation conversation. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. How are 'DOMINIUM RUN ROOT' and 'DOMINIUM HOME' finalized? All platform interaction must go through dsys. (FACT).

The governing constraint is practical rather than cosmetic. Do not perform silent migrations or compatibility reinterpretation. Scope: single-source dependency map and forbidden edges for Dominium/Domino. Scope: canonical statement of identity and intent. An Assembly capable of submitting Intents under AuthorityContext and LawProfile constraints.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. A term or pattern forbidden for use in code identifiers. Game code depends on logical components, not layout or memory order. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. Verify every implementation, platform, tooling, release, and queue claim against current repo artifacts.

The risk is that successful automation can look like authority, especially when it generates polished reports or books.

For aide, codex, automation, and repo governance, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around AIDE, Codex, task governance, validation points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use aide, codex, automation, and repo governance as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for AIDE, Codex, task governance, validation, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Constitutional conflicts are resolved by this document, not by prompt convenience. Inconsistent clients, shard drift, and audit failure. This is the full canonical glossary v1.0.0. If terminology conflicts with local or legacy docs, this glossary wins. Crosses unresolved or quarantined ownership areas.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Speculative future feature not yet implemented. The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would. A third unresolved goal is confirming whether command graph, UI IR, binding validation, zero-pack tests, RepoX changelog display, and BUILD-ID mismatch refusal are implemented. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. Verify latest live repo gate state. If fast-strict/RepoX still fails on stale evidence or launcher marker debt, fix that first. If normal gates are clean, continue with 'PROJECTION-CONFORMANCE-01'. Later live-state checks suggested that the visible root problem was largely resolved and that the project had moved to the next problem: semantic governance inside the approved roots. Harness engineering became the major doctrine. The visible rationale was that agents perform better when the repo provides stable context, reliable commands, test recipes, permission models, worktree isolation, memory, and evidence. This moved the. Early in the conversation, some ideas were explored that later became obsolete. For example, there was an early suggestion that Visual Studio and Xcode projects could be hand-authored as canonical shells. That was later superseded by the user's applied.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on. Affects: UI structure, module order, implementation prompts. The 'src' issue illustrates the reasoning. A generic 'src' root is not bad in all projects, but in Dominium it hides ownership and allowed Codex/humans to create shadow implementations. Therefore, the policy became ownership-rooted layout rather than.

A later task touching this area should therefore state its validation path before it starts changing behavior. The discussion then mapped these ideas onto Dominium's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX, BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We. Dominium/Domino Architecture and Codex Prompt Roadmap - Conversation Reader. A 14-prompt Codex implementation roadmap exists.

The right use of automation is to expose evidence and perform bounded edits while leaving semantic authority with the repo.

4.4 13. Documentation, Archive, and Conversation Corpus

The documentation archive is a memory system.

Current docs-corpus outputs inventory and classify the docs tree, while the conversation corpus preserves the project's historical design conversations.

The current material supports that picture in several ways. Travel scheduling, replay, and law decisions become nondeterministic. Dependency direction is authoritative and must not be inverted. Law admission gates sit before scheduling, before execution, and before commit. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based.

A second current-thread is the project's insistence that roles remain explicit. Scope: hard invariants for all systems, documentation, and data. Reachability lies, bypassed costs, and broken domain law. Invariant: No authority without capability and law. Invariant: Authority gates actions only, never visibility (AUTH3-AUTH-001).

Current documentation also narrows the interpretation of the theme. Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003). Invariant: Authority upgrades/downgrades do not mutate state (AUTH3-UPGRADE-007). Invariant: Saves are tagged by authority scope (AUTH3-SAVE-008). Dominium/Domino is

a deterministic world runtime and law-governed simulation OS.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. A law-governed simulation OS where authority is data-defined. A multi-scale reality model with explicit existence/refinement and visitability. God-mode admin tools (law-gated and audited). There are no game modes; behavior is law + capability + policy.

The archive contains synthesis, decisions, promotion candidates, contradictions, wiki navigation, reader pages, manifests, and audit material.

The conversation corpus adds more texture. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Reader Brief - Dominion Canon, Repository Alignment, and Portability Doctrine.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work. SELF-AUDIT AND REVISION - Dominion Canon, Repository Alignment, and Portability Doctrine. Separated repo docs, schemas, code, and tests in maturity claims. Reliability rating: 4/5 with explicit caveats.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. How should public API/ABI boundaries be written? The conversation established that Dominion should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts. The central architectural conclusion is that Domino should be a framework: it defines stable contracts, service APIs, provider manifests, profiles, capability/refusal law, conformance tests, and deterministic simulation boundaries. A Domino engine. Raylib as provider suite, not engine law.

The archive also records how the project repeatedly returned to the same pressure points from different directions. C17 + C++17 mainline; C-compatible ABI; deterministic fixed-width law; 64-bit source-native; little-endian; providers not identity; Workbench not authority; Foundation blocker first. Added a reuse-proof workstream rather than claiming reuse is already proven. Calendars are pure renderers and never affect physics. Adopt base-16 spatial ladder from subnano to surface.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors. Calendars are pure renderers and never affect physics.

Other decisions remain tentative because the source material records review posture rather than final promotion. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. Final support tiers are user-defined and must be preserved. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. Domain permissions + existence/refinement - visitability decision. Which decisions are confirmed versus recommendations? Review decisions with uncertainty labels, then formalise the vertical slice.

The governing constraint is practical rather than cosmetic. Scope: single-source dependency map and forbidden edges for Dominion/Domino. Scope: canonical statement of identity and intent. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed.

Game code depends on logical components, not layout or memory order. Save/replay integrity depends on identical macro transitions.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Do not perform silent migrations or compatibility reinterpretation. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days. Mass moves or broad rewrites can break build, tests, imports, packaging, and compatibility. Use small gated migrations.

The book's job is to turn that mass into an intelligible map without making old claims current.

For documentation, archive, and conversation corpus, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around docs archive, conversation corpus, source archaeology points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use documentation, archive, and conversation corpus as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for docs archive, conversation corpus, source archaeology, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Inconsistent clients, shard drift, and audit failure. Refusal and defer semantics are part of determinism and replay guarantees. Crosses unresolved or quarantined ownership areas. 'archive/': historical, superseded, quarantined, or provenance-retained.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Use negative architecture rules to prevent drift. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. Another tradeoff is between a clean theoretical structure and the current repo's transitional debt. The current ownership-root target is good, but the repo still has cleanup work. The safe path is not broad movement by directory name; it is gated. Included specific risks about overclaiming implementation status. Later live-state checks suggested that the visible root problem was largely resolved and that the project had moved to the next problem: semantic governance inside the approved roots. The main unresolved work is

converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

The conversation corpus is represented here through 173 card-bearing source documents drawn from 761 tracked conversation-archive files. The mined conversation evidence includes 187 decision-oriented observations, 298 specification-like claims, 113 design goals, 312 constraints or prerequisites, and 140 contradiction, risk, or open-question records. Those counts do not make the material authoritative; they show why the archive is useful as a map of design pressure, repeated intent, and unresolved review work.

For ordinary reading, the important content is woven into chapters; for audit, source notes and full appendices remain available.

Chapter 5

Part IV - Simulation and World Model

5.1 14. Reality, Space, Time, and Scale

Reality, space, time, and scale are simulation concepts before they are rendering concepts.

Current doctrine supplies the generic law: deterministic process, authoritative truth, observation, and provenance.

The current material supports that picture in several ways. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Dominium/Domino is a deterministic world runtime and law-governed simulation OS. Travel scheduling, replay, and law decisions become nondeterministic. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence.

A second current-thread is the project's insistence that roles remain explicit. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. A multi-scale reality model with explicit existence/refinement and visitability. Authoritative logic is deterministic and law-gated.

Current documentation also narrows the interpretation of the theme. Micro detail is refined only when required; macro existence is valid. Determinism and replay-first design are mandatory. Replays diverge, cross-shard reconciliation fails, and saves become invalid. It is built to host multi-scale universes with explicit existence, travel,.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. It is not acceptable to fabricate existence or resources. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The entire conversation is anchored by the requirement that Dominium must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern.

The conversations add pressure for large-scale worlds, visitability, temporal resilience, timekeeping, and representations that can refine without breaking truth.

The conversation corpus adds more texture. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable,

and integrated through clear read/write boundaries. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. The entire conversation is anchored by the requirement that *Dominium* must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. *Domino* should store the true universe as seeds, rules, fields, sparse. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; *Domino* as future adapter. Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The strongest creative premise was the robotic seed-civilisation idea. Humans in the past built a robotic mothership, sent it to another solar system, and tasked it with landing on a habitable planet and manufacturing robots to restart civilisation. Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security. The main outcome was a practical doctrine: use commercial engines for what they are good at, but do not delegate the canonical universe state to them. Determinism, resource accounting, fog-of-war authority, sparse planet edits, region handoff. What is *Domino* exactly? Should Unreal be first frontend? Which language/math stack should *DominiumSim* use? What terrain model is best? How exact must collapsed simulation be? What first MMO scale target is realistic?

The archive also records how the project repeatedly returned to the same pressure points from different directions. The answer challenged the framing. It said that UE6 should not be treated as a present production target because public UE6 tooling and detailed requirements were not established in the visible sources. It recommended building in current Unreal. The final user-confirmed world model uses a toroidal horizontal SurfaceGrid of 2^{24} metres per axis, one vertical Page of 4096 metres, 16 metre cubic chunks, runtime Q16.16 coordinates with 8-bit Segment addressing, and Q4.12 chunk-local save. The practical purpose is simple: a future reader should be able to open this one file and quickly understand the substance of the conversation without needing to reread the whole transcript. It is not a replacement for the structured package; it is the. The central conclusion was that no commercial engine can provide this whole stack out of the box. Unreal Engine can be useful, especially as a renderer, editor, visual client, construction-preview tool, and asset pipeline. UE6 may eventually be useful if.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process.

Other decisions remain tentative because the source material records review posture rather than final promotion. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. *Domino* should store the true universe as seeds, rules, fields, sparse. Main decision candidates: custom deterministic core; Unreal as frontend

only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Save/replay integrity depends on identical macro transitions. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. The mothership solves several design problems at once. It explains why players have a HUD, blueprints, recipes, construction guidance, and advanced technical knowledge. Players do not need to reinvent CPUs, railway gauges, smelting, computing, or.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. The main outcome is not that Dominion needs a conceptual rewrite. The outcome is that Dominion's existing model is directionally correct, but needs boundary hardening: audit all DSYS backend timers, freeze ACT units and serialization semantics, prevent. AIDE was positioned as a build intelligence and evidence layer. It should probe the machine, explain which tuples are available or blocked, generate local presets, run tuple verification, and record evidence. XStack should remain umbrella orchestration. The conversation also produced a practical near-term direction: do not wait for UE6. Build a deterministic headless prototype first. Keep gameplay rules, world state, resource accounting, factory logic, terrain edits, fog-of-war authority, and.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. This is a recommendation, not a user-confirmed decision. It requires current verification and prototype tests before being formalised. The first real milestone should be small. Prove spawn, survey, build, mine, refine, fabricate, and create a new spawn point before scaling to planets, cities, and MMO systems. The central proposal from that first phase was that Dominion should follow a kernel-first deterministic simulation route. In plain terms, this means starting with the smallest reliable simulation core: time, deterministic math, random number. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days.

The unresolved issue is how much of that long-horizon world model is current spec versus design ambition.

For reality, space, time, and scale, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around reality model, space, scale, time points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the

repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use reality, space, time, and scale as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for reality model, space, scale, time, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. 'archive/': historical, superseded, quarantined, or provenance-retained. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer. The current repo should not be overstated as a complete runtime implementation of all v1 goals. The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would.

The downstream effect is that future work has to account for more than the immediate feature request. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on.

Future domain work should start by identifying the authority of each claim before it is attached to gameplay or renderer behavior.

5.2 15. World Generation, Celestial Systems, and Domains

World generation and celestial systems recur because the project wants large worlds that remain lawful and inspectable.

Current authority supports deterministic composition and data-driven domain boundaries but does not authorize broad worldgen implementation.

The current material supports that picture in several ways. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. This authorization remains narrow. It does not authorize broad Workbench UI, gameplay, renderer, native GUI, runtime provider expansion, package runtime, release publication, or domain feature work. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Dominion is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition.

A second current-thread is the project's insistence that roles remain explicit. Packs provide capabilities, data, and assets; executables do not embed content. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Invariant: No authority without capability and law. Dominion/Domino is a deterministic world runtime and law-governed simulation OS.

Current documentation also narrows the interpretation of the theme. A multi-scale reality model with explicit existence/refinement and visitability. There are no game modes; behavior is law + capability + policy. Micro detail is refined only when required; macro existence is valid. Admin power is capability- and law-gated, audited, and fork-aware.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. It is built to host multi-scale universes with explicit existence, travel,. Content-agnostic executables: all meaning comes from packs resolved by UPS. It is not acceptable to fabricate existence or resources. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries.

The archive discusses astronomy, planetary systems, domain packs, terrain, scale, and source verification as future design material.

The conversation corpus adds more texture. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. The practical purpose is simple: a future reader should be able to open this one file and quickly understand the substance of the conversation without needing to reread the whole transcript. It is not a replacement for the structured package; it is the.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. The conversation then expanded from one subsystem-resources, items, recipes, entities, and extraction mechanics-into almost the entire engine and product ecosystem. We discussed how resource fields should work, how players should build factories, how. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance lan-

guage. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Write the implementation prompt for public surface registry.

Several decisions are folded into this topic rather than standing apart from it. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. Calendars are pure renderers and never affect physics.

Other decisions remain tentative because the source material records review posture rather than final promotion. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Executables do not embed content; packs are external and optional. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. Do not start broad Workbench UI, renderer, native GUI, package runtime, provider runtime, module loader, gameplay, or public release yet.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. The main tradeoff was between ambition and discipline. Dominion's vision is huge: universe-scale exploration,

arbitrary invention, civilization simulation, institutions, knowledge propagation, and moddable alternate realities. But the repo and doctrine. The project includes planets, civilisations, terraforming, factories, megaprojects, and MMO persistence. The immediate risk is trying to build all of it before proving the smallest fun loop. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days. Mass moves or broad rewrites can break build, tests, imports, packaging, and compatibility. Use small gated migrations.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. The mothership solves several design problems at once. It explains why players have a HUD, blueprints, recipes, construction guidance, and advanced technical knowledge. Players do not need to reinvent CPUs, railway gauges, smelting, computing, or. The immediate plan is not more doctrine. The next active task is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01. It should classify the 358 dependency-direction violations and 38 warnings, repair real boundary leaks, add only narrow justified exceptions. The actual repo was not inspected. The exact accepted directory structure, API policy, DDAP profile, compatibility promises, and first pilot module remain unresolved.

The difficult part is keeping scientific ambition, authored content, and deterministic constraints aligned.

For world generation, celestial systems, and domains, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around worldgen, astronomy, planetary domains points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use world generation, celestial systems, and domains as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for worldgen, astronomy, planetary domains, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod

loading, multiplayer.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred. 7. The current code is promising but not yet the full vision. It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired. The main unresolved goals are backend audit, ACT serialization audit, civil/astronomical projection design, and verification of external platform facts.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

The next useful work is not implementation but source verification, domain contract scoping, and decision triage.

5.3 16. Civilization, Economy, Logistics, and Institutions

Civilization-scale simulation is one of the clearest ambitions in the corpus.

Current orientation names production, logistics, economics, settlement, trust, communication, and institutional power as intended emergent phenomena.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Dominion/Domino is a deterministic world runtime and law-governed simulation OS.

A second current-thread is the project's insistence that roles remain explicit. A multi-scale reality model with explicit existence/refinement and visitability. Micro detail is refined only when required; macro existence is valid. Runtime behavior MUST resolve from profile data ('ExperienceProfile', 'LawProfile', 'ParameterBundle', optional 'MissionSpec' constraints). Systems MUST remain lawful with optional subsystems absent (for example: no AI, no economy, no war).

Current documentation also narrows the interpretation of the theme. All runtime-impacting schemas MUST declare 'schema id', 'schema version', and 'stability'. Deprecated terms must not be introduced in new runtime logic. Foundation Lock means the governance spine exists, validates in required scope, is documented, and is integrated with the normal fast strict proof gate. It allows only narrow product slices that obey the public surface, command, diagnostics, artifact. It is built to host multi-scale universes with explicit existence, travel,

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of

authority while the project grows. It is not acceptable to fabricate existence or resources. This authorization remains narrow. It does not authorize broad Workbench UI, gameplay, renderer, native GUI, runtime provider expansion, package runtime, release publication, or domain feature work. Current repo truth: Dominion is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical.

Conversation material adds seed civilizations, infrastructure, signals, institutions, economies, and social-scale feedback loops.

The conversation corpus adds more texture. Current repo truth: Dominion is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical. The main outcome was a practical doctrine: use commercial engines for what they are good at, but do not delegate the canonical universe state to them. Determinism, resource accounting, fog-of-war authority, sparse planet edits, region handoff. Calendars are pure renderers and never affect physics.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be.

The archive also records how the project repeatedly returned to the same pressure points from different directions. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into. Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work.

Several decisions are folded into this topic rather than standing apart from it. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. The key technical conclusion was that the world cannot

be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse.

Other decisions remain tentative because the source material records review posture rather than final promotion. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are. Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. The idea is attractive because it supports broad distribution and long-term extensibility. But it is not a final decision. Dynamic plugins introduce ABI, packaging, security, and platform restrictions. Static registration may be simpler. A hybrid may be. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto. The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into.

The governing constraint is practical rather than cosmetic. Foundation Lock does not mean full CTest is green, every compatibility corpus exists, every provider is implemented, every runtime trust rule is enforced, Workbench UI exists, or broad feature work is open. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Current blocked queue scopes: 'broad workbench ui', 'gameplay', 'native gui', 'package runtime', 'provider runtime', 'release publication', 'renderer implementation', 'runtime module loader'. 'platform' (39 conversations): runtime shell, platform adapter, renderer boundary, operating-system support, and portability. Sources: Dominion Advanced Simulation and Infrastructure.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. It does not open blocked renderer, gameplay, provider runtime, package runtime, broad Workbench UI, native GUI, or release publication work. The main tradeoff was between ambition and discipline. Dominion's vision is huge: universe-scale exploration, arbitrary invention, civilization simulation, institutions, knowledge propagation, and moddable alternate realities. But the repo and doctrine. The central proposal from that first phase was that Dominion should follow a kernel-first deterministic simulation route. In plain terms, this means starting with the smallest reliable simulation core: time, deterministic math, random number. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice. The mothership solves several design problems at once. It explains why players have a HUD, blueprints, recipes, construction guidance, and advanced technical knowledge. Players do not need to reinvent CPUs, railway gauges, smelting, computing, or. AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. The main outcome is not that Dominion needs a conceptual rewrite. The outcome is that Dominion's existing model is directionally correct, but needs boundary hardening: audit all DSYS backend timers, freeze ACT units and serialization semantics, prevent.

Those sources are rich, but they remain mostly design context until domain tasks are opened.

For civilization, economy, logistics, and institutions, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around civilization simulation, economy, logistics, institutions points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use civilization, economy, logistics, and institutions as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for civilization simulation, economy, logistics, institutions, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate. The exact module architecture remains unresolved. The exact platform/renderer support matrix remains unresolved. The exact relationship to the current repository remains unresolved because the old snapshot has not been verified. The exact setup trust. The current repo should not be overstated as a complete runtime implementation of all v1 goals. The unique contribution is the synthesis: Dominium should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Proper time matters for local clocks, aging, and relativistic effects, but making it the basis of civil calendars would create frame-dependent disagreements and log problems. It was rejected as authoritative calendar time. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems. 7. The current code is promising but not yet the full vision. It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired. The main unresolved goals are backend audit, ACT serialization audit, civil/astronomical projection design, and verification of external platform facts. This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation. The discussion then mapped these ideas onto Dominion's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX, BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We.

The value today is conceptual: they show what kinds of consequences the lower-level architecture must eventually be able to support.

5.4 17. Content, Providers, Modding, and Data-Driven Extension

Content extension is supposed to be governed, not magical.

Current doctrine favors packs, registries, explicit capabilities, compatibility rules, and deterministic refusal over hidden fallback.

The current material supports that picture in several ways. The project is about invention, production, logistics, economics, settlement, trust, communication, and institutional power emerging from lawful simulation rather than scripted outcomes. Commands, processes, packs, capabilities, diagnostics, evidence. Content is authored data, not runtime law. Packs and registries describe optional content, capabilities, compatibility, activation, and distribution surfaces. Missing optional content must degrade or refuse explicitly. Packs provide capabilities, data, and assets; executables do not embed content. Invariant: No authority without capability and law.

A second current-thread is the project's insistence that roles remain explicit. There are no game modes; behavior is law + capability + policy. Admin power is capability- and law-gated, audited, and fork-aware. Dominion is the official game, product, and domain layer on top of Domino. It defines rules, process meaning, law targets, domain interpretation, authored content use, and product composition. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence.

Current documentation also narrows the interpretation of the theme. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout. When behavior or compatibility meaning changes, update the relevant contract surface and proof in the same task. Scope: index of frozen and evolving constitutional surfaces. This index is the navigation hub for stability surfaces. "FROZEN" means the.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Document defines a contract that must not change lightly. "EVOLVING" means the. Schemas are the authoritative data-shape contracts. Start here. Reachability lies, bypassed costs, and broken domain law. A multi-scale reality model with explicit existence/refinement and visitability.

The archive expands this into providers, package runtime, open-source provider choices, authored content workflows, and modding surfaces.

The conversation corpus adds more texture. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt. Write the implementation prompt for public surface registry. The central architectural conclusion is that Domino should be a framework: it defines stable contracts, service APIs, provider manifests, profiles, capability/refusal law, conformance tests, and deterministic simulation boundaries. A Domino engine. Top tasks: verify repo baseline, draft framework ABI, add provider manifests, add forbidden include validator,

implement null providers, implement raylib providers, create profiles, and define work-lease/CAD schemas.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Main open questions: dependency versions, C17/C++17 repo state, Lua version, framework ABI, profile order, deterministic class, license policy, CAD graph scope, and distributed compute verification. C17 + C++17 mainline; C-compatible ABI; deterministic fixed-width law; 64-bit source-native; little-endian; providers not identity; Workbench not authority; Foundation blocker first. The exact first implementation branch, versions, and dependency pins are unresolved. The exact Domino Framework ABI is unresolved. The exact Lua version is unresolved. The current repository baseline must be verified. The formal policy for. The user's underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of Dominion: a deterministic, moddable, inspectable, replayable universe platform where client, server.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Write the implementation prompt for tuple-driven build contracts. Dmem and dserialize APIs are inconsistent across drafts. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and. Reader Brief - Dominion Canon, Repository Alignment, and Portability Doctrine.

Several decisions are folded into this topic rather than standing apart from it. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels.

Other decisions remain tentative because the source material records review posture rather than final promotion. Domain permissions + existence/refinement - visitability decision. Chosen provider capabilities flow into CAP-NEG and may drive explicit degrade/refuse outcomes. 'session begin' binds the connection to the chosen contract bundle hash and pack lock hash.

The governing constraint is practical rather than cosmetic. Foundation Lock does not mean full CTest is green, every compatibility corpus exists, every provider is implemented, every runtime trust rule is enforced, Workbench UI exists, or broad feature work is open. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Do not perform silent migrations or compatibility reinterpretation. Executables do not embed content; packs are external and optional.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Mass moves or broad rewrites can break build, tests, imports, packaging, and compatibility. Use small gated migrations. The actual repo was not inspected. The exact accepted directory structure, API policy, DDAP profile, compatibility promises, and first pilot module remain unresolved. AIDE itself also changed direction.

It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into. The immediate next action is not more architecture brainstorming. The latest project task/review packets say Foundation Lock is blocked because dependency-direction strict validation reports 358 violations and 38 warnings. The next task is.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days. Manual and queued work both flow through jobs. 07 Verification and audit: Sections 32-34 and verification queue. The immediate plan is not more doctrine. The next active task is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01 . It should classify the 358 dependency-direction violations and 38 warnings, repair real boundary leaks, add only narrow justified exceptions.

Much of that expansion is blocked or not yet formalized.

For content, providers, modding, and data-driven extension, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around content packs, providers, modding, extension points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use content, providers, modding, and data-driven extension as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for content packs, providers, modding, extension, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed . A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer. The current repo should not be overstated as a complete runtime implementation of

all v1 goals. Use negative architecture rules to prevent drift. The unique contribution is the synthesis: Dominium should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. DDAP/DIL is proposed, not yet user-ratified. Another tradeoff is between a clean theoretical structure and the current repo's transitional debt. The current ownership-root target is good, but the repo still has cleanup work. The safe path is not broad movement by directory name; it is gated. Included specific risks about overclaiming implementation status. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt.

The immediate use is to separate already-current pack doctrine from future provider and package-runtime proposals.

Chapter 6

Part V - Decisions and Roadmap

6.1 18. What Is Settled

Several principles are settled strongly enough to shape every future task.

Authority order, process-only mutation, truth/perception/render separation, profiles over mode flags, no hidden fallback, pack-driven integration, and explicit validation are recurring current truths.

The current material supports that picture in several ways. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. Dependency direction is authoritative and must not be inverted. Law admission gates sit before scheduling, before execution, and before commit.

A second current-thread is the project's insistence that roles remain explicit. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Scope: hard invariants for all systems, documentation, and data. Reachability lies, bypassed costs, and broken domain law. Invariant: No authority without capability and law.

Current documentation also narrows the interpretation of the theme. Invariant: Authority gates actions only, never visibility (AUTH3-AUTH-001). Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003). Invariant: Authority upgrades/downgrades do not mutate state (AUTH3-UPGRADE-007). Invariant: Saves are tagged by authority scope (AUTH3-SAVE-008).

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. A law-governed simulation OS where authority is data-defined. A multi-scale reality model with explicit existence/refinement and visitability. God-mode admin tools (law-gated and audited). There are no game modes; behavior is law + capability + policy.

The conversation archive often reinforces these decisions, which is useful but not necessary for their authority.

The conversation corpus adds more texture. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR. Current repo truth: Dominium is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. Domino is the deterministic substrate. Dominium is the official game, product, and domain layer. Workbench is a governed validation and inspection environment. Setup and launcher compose product instances. Tools validate, generate, audit, and migrate. The archive describes Dominium as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. This atlas is a reader map. It is not canon and does not promote archived conversation claims. This map combines

current repo orientation with conversation-derived design intent. When they differ, current repo authority wins. ”: What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators. What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? The entire conversation is anchored by the requirement that Dominium must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern. Use this page as the shortest path through the conversation-corpus adjudication surfaces.

The archive also records how the project repeatedly returned to the same pressure points from different directions. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors. The most important repo-supported decision is that Dominium authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. ”: What disposition should be chosen for this unresolved claim: Future work must define the client’s allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? This context matters because the rest of the conversation happened inside those constraints. The launcher could not become a dumping ground for OS-specific behavior, sim mutation, renderer decisions, or nondeterministic game-state changes. It had to fit.

Other decisions remain tentative because the source material records review posture rather than final promotion. Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators. Use: design intent, review backlog, decision and promotion candidates. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. The most important repo-supported decision is that Dominium authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. A large set of prompts was then generated to lock down architecture, determinism, performance, schema governance, rendering, epistemic UI, sharding, interest sets, and fidelity projection. This became the Phase 1 hardening layer. Additional audit prompts. The visible conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority. There was also an early suggestion to create a ’DUI’ facade, a Dominium UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form.

The governing constraint is practical rather than cosmetic. Scope: single-source dependency map and forbidden edges for Dominium/Domino. Scope: canonical statement of identity and intent. An Assembly capable of submitting Intents under AuthorityContext and LawProfile constraints. Responsibilities and forbidden dependencies (quick map).

Those constraints matter because they prevent future work from using the book as an implementation shortcut.

Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed. Client and renderer project data and issue intents; they do not author authoritative outcomes.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. A term or pattern forbidden for use in code identifiers. Game code depends on logical components, not layout or memory order. Save/replay integrity depends on identical macro transitions. Delegated execution requires deterministic proof artifacts before commit.

The settled material should be used to simplify future review: if a proposal violates one of these principles, the burden is on the proposal.

For what is settled, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around settled decisions, current repo truth points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use what is settled as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for settled decisions, current repo truth, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Inconsistent clients, shard drift, and audit failure. Constitutional conflicts are resolved by this document, not by prompt convenience. This is the full canonical glossary v1.0.0. If terminology conflicts with local or legacy docs, this glossary wins. Refusal and defer semantics are part of determinism and replay guarantees.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Speculative future feature not yet implemented. Continue documentation and evidence updates that keep remaining warnings and full-gate debt visible. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Use negative architecture rules to prevent drift.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would. A third unresolved goal is confirming whether command graph, UI IR, binding validation, zero-pack tests, RepoX changelog display, and BUILD-ID mismatch refusal are implemented. Source rules: labels used: "FACT", "INFERENCE", "UNCERTAIN / UNVERIFIED", "PROJECT-CONTEXT" conflict rules. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none

of them should invent. Affects: UI structure, module order, implementation prompts. The 'src' issue illustrates the reasoning. A generic 'src' root is not bad in all projects, but in Dominion it hides ownership and allowed Codex/humans to create shadow implementations. Therefore, the policy became ownership-rooted layout rather than. 4. Conceptual artifacts produced in chat. These include proposed architectures, matrices, lane plans, Workbench modules, Interface Operating Layer, boot-to-replay MVP, and structured roadmaps.

A later task touching this area should therefore state its validation path before it starts changing behavior. The discussion then mapped these ideas onto Dominion's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX, BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We. Tool Editor eventually self-hosts and edits itself/other tools. Dominion/Domino Architecture and Codex Prompt Roadmap - Conversation Reader. 6. Long-term live operations remain future work.

The next step is to turn aligned archive phrasing into carefully scoped clarifications only where current docs need them.

6.2 19. What Is Still Open

The open questions are not a defect; they are the reason the archive exists as evidence instead of doctrine.

Docs-corpus and conversation dockets identify user decisions, future queue decisions, and source verification needs.

The current material supports that picture in several ways. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. Scope: index of frozen and evolving constitutional surfaces. This index is the navigation hub for stability surfaces. "FROZEN" means the.

A second current-thread is the project's insistence that roles remain explicit. Document defines a contract that must not change lightly. "EVOLVING" means the. Schemas are the authoritative data-shape contracts. Start here. 'FOUNDATION-CLOSEOUT-02' reran the Foundation Lock gate after 'FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01' and closed the lock with warnings. 'PORTABILITY-ARCH-POLICY-02' has since completed the first post-closeout hardening follow-up. All required files for the 15 Foundation Lock layers are present.

Current documentation also narrows the interpretation of the theme. Current partially aligned to the Constitutional Architecture and release-freeze documentation set. The simulation kernel is the source of truth; perception layers are derived. Objective snapshots record authoritative state; subjective snapshots record views. Determinism and replay-first design are mandatory.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Document is binding but still expected to grow or sharpen. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators. Use: design intent, review backlog, decision and promotion candidates.

The archive contains many plausible answers, but a plausible answer is not the same as a repo decision.

The conversation corpus adds more texture. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators. What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? The entire conversation is anchored by the requirement that Dominion must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Use this page as the shortest path through the conversation-corpus adjudication surfaces. Verify latest queue. If unchanged, proceed with 'PRESENTATION-CONTRACT-01'; then draft 'UNIVERSE-EXPLORER-CONTRACT-01'. Calendars are pure renderers and never affect physics. Adopt base-16 spatial ladder from subnano to surface.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. Final support tiers are user-defined and must be preserved. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. Which repo files supported the Dominion conclusion?

The archive also records how the project repeatedly returned to the same pressure points from different directions. The final state of the conversation is therefore: the architecture and implementation-roadmap discussion has been preserved, but the actual next work is elsewhere-especially the Path D advanced simulation conversation. Most important: actual repo structure, existing GPT-5.2 refactor plan contents, final Q formats, orientation representation, grid assumptions in current code, microsegment length, packing VM instruction set, bridge/tunnel ownership, and checksum rules. How are 'DOMINIUM RUN ROOT' and 'DOMINIUM HOME' finalized? Read file 01 first if you want to understand what happened. Read file 04 if you need tasks and decisions. Read file 05 if merging into a master Project Spec Book. Read file 07 before treating external support claims as implementation truth.

Several decisions are folded into this topic rather than standing apart from it. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. ": What disposition should be chosen for this unresolved claim: Future work must define the client's allowed dependency surface and whether it can access any simulation-adjacent prediction layer.? Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators.

Other decisions remain tentative because the source material records review posture rather than final promotion. Use: design intent, review backlog, decision and promotion candidates. Calendars are pure renderers and never affect physics. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. Final support tiers are user-defined and must be preserved. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. Which repo files supported the Dominion conclusion? The final state of the conversation is therefore: the architecture and implementation-roadmap discussion has been preserved, but the actual next work is elsewhere-especially the Path D advanced simulation conversation.

The governing constraint is practical rather than cosmetic. Foundation Lock does not mean full CTest is green, every compatibility corpus exists, every provider is implemented, every runtime trust rule is enforced, Workbench UI exists, or broad feature work is open. Keep Queue B hardening planned but not a substitute for the blocker repair. 'FOUNDATION-CLOSEOUT-01' was blocked because dependency-direction strict reported '358' violations and '38' warnings. Items touching blocked queue scope must remain explanatory or deferred; no implementation scope is opened.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Current blocked queue scopes: 'broad workbench ui', 'gameplay', 'native gui', 'package runtime', 'provider runtime', 'release publication', 'renderer implementation', 'runtime module loader'. Do not open blocked queue scopes from old conversations. The current queue blocks these broad work areas. Docs-corpus outputs record them for navigation only. Use generated contradictions and promotion queues to plan later tasks.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. This queue records later review candidates. It does not apply any live-doc patch. It does not open blocked renderer, gameplay, provider runtime, package runtime, broad Workbench UI, native

GUI, or release publication work. Do not promote archive claims without a later reviewed task. Any Wave 1 candidate before source text and target docs are inspected together.

The work ahead is to separate questions that can be answered by docs hygiene from questions that need queue scope or user authority.

For what is still open, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around open decisions, user questions, future queue points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use what is still open as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for open decisions, user questions, future queue, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Continue documentation and evidence updates that keep remaining warnings and full-gate debt visible. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred. The following were not finished or intentionally deferred. The reason these were deferred is not that they are unimportant. They were deferred because the local playable baseline path is not yet reliable. Feature work before baseline proof would increase uncertainty and project risk.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. How can Dominion grow for years or decades without collapsing into special cases, silent semantic drift, contradictory abstractions, or unmaintainable simulation layers? Proper time matters for local clocks, aging, and relativistic effects, but making it the basis of civil calendars would create frame-dependent disagreements and log problems. It was rejected as authoritative calendar time. DDAP/DIL is proposed, not yet user-ratified. Codex prompts were generated; application status is unknown.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The main unresolved goals are backend audit, ACT serialization audit, civil/astronomical projection design, and verification of external platform facts. UNCERTAIN / UNVERIFIED The exact VM is unresolved. The most important caveat is that actual repo state is unknown. 'UNCERTAIN / UNVERIFIED:' Whether the project has similarly named files or modules is unknown.

The downstream effect is that future work has to account for more than the immediate feature request. 4. Conceptual artifacts produced in chat. These include proposed architectures, matrices, lane plans, Workbench modules, Interface Operating Layer, boot-to-replay MVP, and structured roadmaps. Tool Editor eventually self-hosts and edits itself/other tools. Dominion/Domino Architecture and Codex Prompt Roadmap - Conversation Reader. 6. Long-term live operations remain future work.

This book should help the reader recognize the question before trying to solve it.

6.3 20. What Is Blocked

Blocked scope is as important as active scope.

The current queue does not authorize broad Workbench UI, renderer implementation, native GUI, provider runtime, package runtime, gameplay, or release publication.

The current material supports that picture in several ways. Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003). The archive describes Dominion as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not. The conversations strongly prefer a rich Workbench and eventual editor/operator surfaces. Current authority is narrower: broad Workbench UI, renderer implementation, and native GUI remain blocked. Therefore synthesis may describe intent, but follow-up. The structured permission context required for all Process execution. Includes authority origin, law profile id, entitlements, epistemic scope, and privilege level.

A second current-thread is the project's insistence that roles remain explicit. It is not a generic rendering or physics framework; rendering is derived and. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts. The user's central concern was that the project must be built to last. The desired system is not a throwaway indie codebase. It should be portable, modular, extensible, reusable for different games on the same Domino engine, reusable for other engine.

Current documentation also narrows the interpretation of the theme. The central conclusion was that no commercial engine can provide this whole stack out of the box. Unreal Engine can be useful, especially as a renderer, editor, visual client, construction-preview tool, and asset pipeline. UE6 may eventually be useful if. Renderer and platform are strictly separate from sim. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. Raylib as provider suite, not engine law.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. Ask about provider wedge, robot OS wedge, renderer plan, Workbench UI authoring, robotic seed-civilisation mechanics, UI customization security, or spec-book merge. This is not a verbatim transcript. It is a synthesized, human-readable report intended for project continuity. The most important architecture pattern is: generic engine primitives, not game semantics. Systems communicate through typed packets, fields, events, messages, observations, intents, actions, deltas, ports, probes, and canonical graphs. Mutation happens. The strict source layout validators remain the repo-level proof floor. POST-CONVERGE-10 added a tuple build contract and machine probe. POST-CONVERGE-10B confirmed that Visual Studio 2022/MSVC v143 is detected. POST-CONVERGE-10C fixed the stale.

Older conversations often speak about those areas with enthusiasm because they preserve product intent.

The conversation corpus adds more texture. The archive describes Dominion as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not. The conversations strongly prefer a rich Workbench and eventual editor/operator surfaces. Current authority is narrower: broad Workbench UI, renderer implementation, and native GUI remain blocked. Therefore synthesis may describe intent, but follow-up. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts.

The strongest historical value is not that every claim is current, but that repeated concerns become visible.

The user's central concern was that the project must be built to last. The desired system is not a throwaway indie codebase. It should be portable, modular, extensible, reusable for different games on the same Domino engine, reusable for other engine. The central conclusion was that no commercial engine can provide this whole stack out of the box. Unreal Engine can be useful, especially as a renderer, editor, visual client, construction-preview tool, and asset pipeline. UE6 may eventually be useful if. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. Raylib as provider suite, not engine law.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The most important architecture pattern is: generic engine primitives, not game semantics. Systems communicate through typed packets, fields, events, messages, observations, intents, actions, deltas, ports, probes, and canonical graphs. Mutation happens.

Several decisions are folded into this topic rather than standing apart from it. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. DIST-5 defines the final user-facing polish bar for 'v0.0.0-mock'. It does not add features. It standardizes help text, menu wording, refusals, and status output across CLI, TUI, rendered UI, and optional native adapters. This pass continued canonical structure convergence after CANON STRUCTURE FINALIZE NOW 01. It was limited to structure, naming, ownership, validator, generated-map, and proof cleanup. It did not implement runtime, product, Workbench, renderer, native.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Current blocked queue scopes: 'broad workbench ui', 'gameplay', 'native gui', 'package runtime', 'provider runtime', 'release publication', 'renderer implementation', 'runtime module loader'. It does not open blocked renderer, gameplay, provider runtime, package runtime, broad Workbench UI, native GUI, or release publication work. Responsibilities and forbidden dependencies (quick map).

Those constraints matter because they prevent future work from using the book as an implementation shortcut. AIDE was positioned as a build intelligence and evidence layer. It should probe the machine, explain which tuples are available or blocked, generate local presets, run tuple verification, and record evidence. XS-tack should remain umbrella orchestration. Verify the latest repo queue and Foundation Lock, then draft 'PRESENTATION-CONTRACT-01' if it is not complete; if it is complete, draft 'UNIVERSE-EXPLORER-CONTRACT-01' as a planning artifact, preserving Workbench projection discipline. 07 verification and audit: self-audit and verification queue. A live repo check during the conversation found Foundation Lock initially blocked by dependency-direction strict validation.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. At first, the proposed idea was a global build number that would increment frequently. That model was later weakened by TESTX, which wanted build-number increments only after test success, and then superseded by the latest upstream BUILD-ID-0 model. The main tradeoff was between ambition and discipline. Dominion's vision is huge: universe-scale exploration, arbitrary invention, civilization simulation, institutions, knowledge propagation, and moddable alternate realities. But the repo and doctrine. The repository can change. The prior GitHub inspection is time-bound and not equivalent to a fresh local checkout. Re-verify before modifying code. Quarantine Reasons: 'file scope ambiguity, near duplicate requires review, phase boundary deferred, requires medium risk batch gate, requires single action full gate'.

That enthusiasm is useful only when it is labelled as future work.

For what is blocked, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around blocked scope, non-goals, queue boundaries points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same

concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use what is blocked as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for blocked scope, non-goals, queue boundaries, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. This was the biggest late-stage correction. It did not throw away the earlier work, but it changed the source hierarchy. Most importantly, it superseded earlier build-number ideas. Future work must now reconcile TESTX, TESTX2, TESTX3, and REPOX with. Later live-state checks suggested that the visible root problem was largely resolved and that the project had moved to the next problem: semantic governance inside the approved roots. Harness engineering became the major doctrine. The visible rationale was that agents perform better when the repo provides stable context, reliable commands, test recipes, permission models, worktree isolation, memory, and evidence. This moved the.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent.

A safe task packet should state blocked areas before it starts describing implementation.

6.4 21. Contradictions and Drift

The corpus preserves disagreement because it preserves history.

Current authority order and snapshot intake rules are what prevent old planning, generated summaries, or conversation claims from overriding present repo truth.

The current material supports that picture in several ways. Run 'AIDE-WORKFLOW-LAW-01' next if the queue remains reconciled after. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice.

A second current-thread is the project's insistence that roles remain explicit. Dependency direction is authoritative and must not be inverted. Law admission gates sit before scheduling, before execution, and before commit. Domain volumes define where reality exists and which laws apply. Domain queries are deterministic, budgeted, and SDF-based. Scope: hard invariants for all systems, documentation, and data.

Current documentation also narrows the interpretation of the theme. Reachability lies, bypassed costs, and broken domain law. Invariant: No authority without capability and law. Invariant: Authority gates actions only, never visibility (AUTH3-AUTH-001). Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003).

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of

authority while the project grows. Invariant: Authority upgrades/downgrades do not mutate state (AUTH3-UPGRADE-007). Invariant: Saves are tagged by authority scope (AUTH3-SAVE-008). A law-governed simulation OS where authority is data-defined. A multi-scale reality model with explicit existence/refinement and visitability.

Contradictions appear around scope, implementation readiness, release state, provider/runtime boundaries, and old architecture language.

The conversation corpus adds more texture. Any claim touching canon, glossary, authority order, contracts, schema law, current queue, release, implementation, or blocked scope. The conversations strongly prefer a rich Workbench and eventual editor/operator surfaces. Current authority is narrower: broad Workbench UI, renderer implementation, and native GUI remain blocked. Therefore synthesis may describe intent, but follow-up. Do not patch canon, contracts, schema, implementation, release, or queue files from this atlas. The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Current repo truth: Dominium is a deterministic, contract-governed simulation game and operating environment built on the Domino deterministic substrate. It is about lawful simulation of invention, production, logistics, economics, settlement, trust. The archive describes Dominium as a long-horizon deterministic simulation product: a game, engine-backed operating environment, validation workbench, content platform, and governance-heavy repository. Across the conversations, the recurring intent is not. This atlas is a reader map. It is not canon and does not promote archived conversation claims. This map combines current repo orientation with conversation-derived design intent. When they differ, current repo authority wins.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. This reader page is derived from archived conversation material. It cannot override canon, contracts, schema law, current queue state, live repo structure, or validated repo artifacts. Use this page as the shortest path through the conversation-corpus adjudication surfaces. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Support canonical surface plus multiple surface instances for shards/mirrors. The most important repo-supported decision is that Dominium authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. The visible conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority. There was also an early suggestion to create a 'DUI' facade, a Dominium UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form.

Several decisions are folded into this topic rather than standing apart from it. Travel scheduling, replay, and law decisions become nondeterministic. Rendering is presentational only and MUST NOT mutate truth or enforce authority decisions. Foundation Lock is the repository decision gate that determines whether Dominium can leave the Foundation Lock queue and begin a first narrow governed product slice. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope.

Other decisions remain tentative because the source material records review posture rather than final promotion. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Support canonical surface plus multiple surface instances for shards/mirrors. The most important repo-supported decision is that Dominium authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. The visible conclusion was that the engine has real deterministic substrate pieces, but these

must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. There was also an early suggestion to create a 'DUI' facade, a Dominion UI abstraction, to support native widgets and fallback rendering. This idea was useful as a conceptual stepping stone but was not ultimately locked as a final requirement in the form. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too. The final plan accepted the newer repo structure is clean enough to stop giant structure prompts. Remaining cleanup should be targeted: stale full-gate tests, pack layout canon, residual taxonomy, AIDE state classification, public header ABI. Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?".

The governing constraint is practical rather than cosmetic. Scope: single-source dependency map and forbidden edges for Dominion/Domino. Scope: canonical statement of identity and intent. An Assembly capable of submitting Intents under AuthorityContext and LawProfile constraints. Keep Queue B hardening planned but not a substitute for the blocker repair.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Responsibilities and forbidden dependencies (quick map). Executables do not embed content; packs are external and optional. Global iteration and implicit background simulation are forbidden. "Convenient" implicit behavior is not allowed.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. A term or pattern forbidden for use in code identifiers. 'FOUNDATION-CLOSEOUT-01' was blocked because dependency-direction strict reported '358' violations and '38' warnings. Game code depends on logical components, not layout or memory order. Save/replay integrity depends on identical macro transitions.

The right response is not to smooth them into a single story, but to classify them, quarantine what needs review, and promote only after targeted decisions.

For contradictions and drift, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around contradictions, stale claims, drift points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use contradictions and drift as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for contradictions, stale claims, drift, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Refusal and defer semantics are part of determinism and replay guarantees. Inconsistent clients, shard drift, and audit

failure. This is the full canonical glossary v1.0.0. If terminology conflicts with local or legacy docs, this glossary wins. Speculative future feature not yet implemented.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. The following were not finished or intentionally deferred. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Use negative architecture rules to prevent drift.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would. A third unresolved goal is confirming whether command graph, UI IR, binding validation, zero-pack tests, RepoX changelog display, and BUILD-ID mismatch refusal are implemented. Source rules: labels used: "FACT", "INFERENCE", "UNCERTAIN / UNVERIFIED", "PROJECT-CONTEXT" conflict rules. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process.

The downstream effect is that future work has to account for more than the immediate feature request. This affects the client, server, launcher, setup, and tools. The client can display and interact; the server can host authority; the launcher can start sessions; setup can install and verify; tools can inspect and audit. But none of them should invent. 4. Conceptual artifacts produced in chat. These include proposed architectures, matrices, lane plans, Workbench modules, Interface Operating Layer, boot-to-replay MVP, and structured roadmaps. Tool Editor eventually self-hosts and edits itself/other tools. 6. Long-term live operations remain future work.

This book treats contradictions as signals for future review rather than failures to hide.

6.5 22. Prerequisites and Sequencing

The project has many desirable destinations, but the sequence matters.

Current state points to narrow command/result and package-mount slices, projection conformance, validation debt, and review work before broad feature expansion.

The current material supports that picture in several ways. Any claim touching canon, glossary, authority order, contracts, schema law, current queue, release, implementation, or blocked scope. Do not patch canon, contracts, schema, implementation, release, or queue files from this atlas. Non-use: canon, contract/schema, implementation, release, or queue override. It does not rewrite canon, contracts, schema, implementation, release, or queue state.

A second current-thread is the project's insistence that roles remain explicit. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. Express behavior composition through profiles, bundles, law surfaces, and explicit constraints. AIDE and Codex are repo/control-plane harnesses and bounded patch execution surfaces. They help operate the repository; they do not replace canon, contracts, validation, or evidence. The active portability floor includes Windows 7 SP1, macOS 10.9.5, and Linux. C++17 language mode is allowed, but public ABI surfaces do not expose C++ classes, STL types, exceptions, RTTI, allocator objects, or compiler object layout.

Current documentation also narrows the interpretation of the theme. Invariant: Demo is an authority profile, not a build (AUTH3-DEMO-003). Current partially aligned to the Constitutional Architecture and release-freeze documentation set. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, bare-bones client shell, product-spine review, presentation contracts. The conversation implies a goal of turning Dominium into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of

authority while the project grows. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too. All platform interaction must go through dsys. (FACT). Final active plan is the 9-prompt pack, not earlier superseded plans. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt.

Historical roadmaps add useful sequencing ideas around Workbench, releases, provider choices, docs promotion, and domain work.

The conversation corpus adds more texture. Any claim touching canon, glossary, authority order, contracts, schema law, current queue, release, implementation, or blocked scope. Do not patch canon, contracts, schema, implementation, release, or queue files from this atlas. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts. The conversation implies a goal of turning Dominion into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too. All platform interaction must go through dsys. (FACT). Final active plan is the 9-prompt pack, not earlier superseded plans. Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. Write the implementation prompt for public surface registry. Write the implementation prompt for tuple-driven build contracts. The conversation established that Dominion should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts. Reader Brief - Dominion XStack Release Identity and Versioning.

The archive also records how the project repeatedly returned to the same pressure points from different directions. What should the manifest schema be? How should capabilities/contracts be named and negotiated? Start with Release Constitution and SemVer Component Inventory. Then define suite policy, build identity, channel/lifecycle, artifact names, manifest schema, target taxonomy, and capabilities. The conversation then expanded from one subsystem-resources, items, recipes, entities, and extraction mechanics-into almost the entire engine and product ecosystem. We discussed how resource fields should work, how players should build factories, how.

Several decisions are folded into this topic rather than standing apart from it. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. The conversation implies a goal of turning Dominion into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later. A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too. All platform interaction must go through dsys. (FACT).

Other decisions remain tentative because the source material records review posture rather than final promotion. Final active plan is the 9-prompt pack, not earlier superseded plans. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. POST-CONVERGE-12 stopped before projection tooling discovery or output generation because POST-CONVERGE-11 is blocked and not accepted as sufficient input for portable projection. The repository is not yet ready for final proof. The build-triggered broader TestX suite still has 140 failures, and RepoX ruleset discovery remains stale.

The governing constraint is practical rather than cosmetic. The current queue does not authorize broad feature

work. Broad Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Current blocked queue scopes: 'broad workbench ui', 'gameplay', 'native gui', 'package runtime', 'provider runtime', 'release publication', 'renderer implementation', 'runtime module loader'. Do not perform silent migrations or compatibility reinterpretation. Items touching blocked queue scope must remain explanatory or deferred; no implementation scope is opened.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not open blocked queue scopes from old conversations. The current queue blocks these broad work areas. Docs-corpus outputs record them for navigation only. Use generated contradictions and promotion queues to plan later tasks. This queue records later review candidates. It does not apply any live-doc patch.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Do not silently rewrite broader planning artifacts just to erase that drift unless a prompt explicitly targets them. It does not open blocked renderer, gameplay, provider runtime, package runtime, broad Workbench UI, native GUI, or release publication work. Verify every implementation, platform, tooling, release, and queue claim against current repo artifacts. AIDE was positioned as a build intelligence and evidence layer. It should probe the machine, explain which tuples are available or blocked, generate local presets, run tuple verification, and record evidence. XStack should remain umbrella orchestration.

The main dependency is authority: future work needs the right queue, target surface, contracts, tests, and evidence.

For prerequisites and sequencing, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around prerequisites, dependencies, future queue points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use prerequisites and sequencing as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for prerequisites, dependencies, future queue, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Crosses unresolved or quarantined ownership areas. Refusal and defer semantics are part of determinism and replay guarantees. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate. Resolve DSYS vs 'domino sys' if conflict appears.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. 11. Open Questions and Unresolved Issues. Unresolved goals include confirming live repo state, executing pending maintenance and projection prompts, building read-only Workbench, implementing provider manifests/fences, building raylib provider wedge, proving Universe Explorer.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The old "UI Editor / Tool Editor" idea was superseded. The preferred model is Dominion Workbench as the production environment for building the project. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred. SDK locking was treated as mandatory. The warning was that allowing a newer Windows SDK to leak into old-floor builds can create symbol/runtime-loader failures and hidden dependency creep. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same.

The downstream effect is that future work has to account for more than the immediate feature request. For governance execution, treat the active executable prompt and this file as the canonical post-'Lambda' governance baseline until downstream planning mirrors are refreshed. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

Good sequencing keeps ambition alive without letting it become uncontrolled scope.

6.6 23. Recommended Next Steps

The next useful move is selective review, not another corpus dump.

Current authority allows narrow governed tasks and derived documentation work; it does not authorize sweeping promotion or implementation.

The current material supports that picture in several ways. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators.

A second current-thread is the project's insistence that roles remain explicit. Use: design intent, review backlog, decision and promotion candidates. The entire conversation is anchored by the requirement that Dominion must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts. Adopt base-16 spatial ladder from subnano to surface.

Current documentation also narrows the interpretation of the theme. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. See through. Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation.

The current repo-backed reading is therefore less about feature breadth than about preserving the shape of authority while the project grows. All platform interaction must go through dsys. (FACT). Final active plan is the 9-prompt pack, not earlier superseded plans. Write the implementation prompt for public surface registry. Write the implementation prompt for tuple-driven build contracts.

The generated corpus, evidence cards, and source notes now provide enough material to choose small follow-up tasks deliberately.

The conversation corpus adds more texture. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators. The entire conversation is anchored by the requirement that Dominion must run deterministically. This means the same initial state and same inputs must produce the same state across machines, compilers, operating systems, and eventually retro and modern. A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts.

The strongest historical value is not that every claim is current, but that repeated concerns become visible. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging.

Across the chat-derived reports, the same theme often appears in product, architecture, and governance language. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation. All platform interaction must go through dsys. (FACT). Final active plan is the 9-prompt pack, not earlier superseded plans. Write the implementation prompt for public surface registry.

The archive also records how the project repeatedly returned to the same pressure points from different directions. Write the implementation prompt for tuple-driven build contracts. The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and. The conversation established that Dominion should be treated as a long-lived deterministic platform: stable contracts, replaceable implementations, ownership-rooted layout, portable projections, and no accidental authority from paths, tools, UIs, or prompts. The strongest creative premise was the robotic seed-civilisation idea. Humans in the past built a robotic mothership, sent it to another solar system, and tasked it with landing on a habitable planet and manufacturing robots to restart civilisation.

Several decisions are folded into this topic rather than standing apart from it. Foundation Lock is the repository decision gate that determines whether Dominion can leave the Foundation Lock queue and begin a first narrow governed product slice. Decision items are review prompts. The default disposition is defer unless a later explicit task opens the scope. The conclusion was not a finalized implementation but a working architecture: each simulation domain should be deterministic, fixed-point, content-driven, versioned, replayable, and integrated through clear read/write boundaries. Safe now: archive reading, decision review, promotion review, reconciliation crosswalks, docs-only microtask preparation, and validators.

Other decisions remain tentative because the source material records review posture rather than final promotion. Use: design intent, review backlog, decision and promotion candidates. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process.

The decision pattern that matters most is the distinction between a design direction that has been preserved and a rule that has actually become binding. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation. All platform interaction must go through dsys. (FACT). Final active plan is the 9-prompt pack, not earlier superseded plans.

The governing constraint is practical rather than cosmetic. Keep Queue B hardening planned but not a substitute for the blocker repair. 'FOUNDATION-CLOSEOUT-01' was blocked because dependency-direction strict reported '358' violations and '38' warnings. The current queue does not authorize broad feature work. Broad

Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked until later reviewed phases. Items touching blocked queue scope must remain explanatory or deferred; no implementation scope is opened.

Those constraints matter because they prevent future work from using the book as an implementation shortcut. Do not open blocked queue scopes from old conversations. The current queue blocks these broad work areas. Docs-corpus outputs record them for navigation only. Use generated contradictions and promotion queues to plan later tasks. This queue records later review candidates. It does not apply any live-doc patch.

The same constraints also protect the reader from treating a polished explanation as a license to bypass queue, contract, or validation gates. Manual and queued work both flow through jobs. 07 Verification and audit: Sections 32-34 and verification queue. File 07 verification/audit: self-audit and verification queue. Verify current repo queue and then draft the next authorized planning artifact.

A good next task should name the claim, source evidence, target document, authority impact, validation level, and non-goals.

For recommended next steps, the useful synthesis is not a catalogue of all preserved claims. It is the shape that remains after current authority, archive memory, and conversation evidence are read together. The recurring material around roadmap, next review actions, promotion triage points to a system that wants broad expressive power, but only when that power can be routed through explicit authority, deterministic behavior, and reviewable evidence.

The archive is most useful here as a record of pressure. It shows where the project kept returning to the same concern, where earlier language was too broad, and where product ambition outpaced queue authority. That historical pressure should inform later review, but it should not erase the difference between a preserved idea and a binding rule.

In practical terms, this chapter asks future work to convert desire into a narrower claim before it changes the repo. A good follow-up would identify the exact target surface, describe the authority it relies on, name the validation evidence it will produce, and say which parts of the larger ambition remain outside scope.

The result is a conservative but useful reading posture: preserve the knowledge, keep the uncertainty visible, and use recommended next steps as a review topic rather than an implementation shortcut. That posture is what lets the project remember the full archive without letting the archive silently become the project.

This is especially important for roadmap, next review actions, promotion triage, where many sources describe future usefulness but only some describe current permission. A reader should therefore separate three things while reading: what is binding now, what the archive helps explain, and what still needs a deliberate review task.

Seen that way, the chapter is not trying to settle every preserved claim. It is trying to make the relevant knowledge usable: enough synthesis to understand the topic, enough restraint to avoid overclaiming, and enough traceability for a reviewer to inspect the source notes when a later decision depends on them.

Where the sources remain unsettled, the uncertainty has to be carried forward rather than hidden. Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred. The reason these were deferred is not that they are unimportant. They were deferred because the local playable baseline path is not yet reliable. Feature work before baseline proof would increase uncertainty and project risk. The unique contribution is the synthesis: Dominion should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism. The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would.

Some conflicts are not disagreements about desire; they are disagreements about authority, timing, or proof. A third unresolved goal is confirming whether command graph, UI IR, binding validation, zero-pack tests, RepoX changelog display, and BUILD-ID mismatch refusal are implemented. Resolve DSYS vs 'domino sys' if conflict appears. 11. Open Questions and Unresolved Issues. Unresolved goals include confirming live repo state,

executing pending maintenance and projection prompts, building read-only Workbench, implementing provider manifests/fences, building raylib provider wedge, proving Universe Explorer.

That distinction matters because a future reviewer can often preserve the ambition while still rejecting the implied implementation step. The strongest technical design pattern was high-fidelity when sensed, collapsed when not sensed. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same. 12. Risks, Failure Modes, and What Future Chats Might Get Wrong. 'PACKAGE-MOUNT-SLICE-01' was reported as complete with 'PASS WITH WARNINGS', and the next intended product-spine task was 'REPLAY-PROOF-SLICE-01'. Verify latest live repo gate state. If fast-strict/RepoX still fails on stale evidence or launcher marker debt, fix that first. If normal gates are clean, continue with 'PROJECTION-CONFORMANCE-01'.

The downstream effect is that future work has to account for more than the immediate feature request. For Wargames, fog and information become observer contexts, belief stores, visibility/occlusion, and communications graphs. Knowledge is simulation state, not UI state. This is essential because fog of war affects what players and AI can know and act on. The 'src' issue illustrates the reasoning. A generic 'src' root is not bad in all projects, but in Dominium it hides ownership and allowed Codex/humans to create shadow implementations. Therefore, the policy became ownership-rooted layout rather than. This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation.

The reader should leave with a map: read the book, inspect source notes where needed, triage decisions, and promote only in narrow waves.

Chapter 7

Appendices

7.1 Appendix A - Decision Index

This compact index names the main decision themes in readable form. Detailed source paths and evidence IDs are intentionally kept in the source notes companion.

Domain permissions + existence/refinement - visitability decision. Travel scheduling, replay, and law decisions become nondeterministic. Calendars are pure renderers and never affect physics. HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. Adopt base-16 spatial ladder from subnano to surface. Store surfaces as sparse microgrids/chunks, not continuous dense arrays. Support canonical surface plus multiple surface instances for shards/mirrors. Final support tiers are user-defined and must be preserved. See through . Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process. Which decisions are confirmed versus recommendations? Review decisions with uncertainty labels, then formalise the vertical slice. The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. Domino should store the true universe as seeds, rules, fields, sparse. This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging. Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels. The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only. Which repo files supported the Dominion conclusion? The answer distinguished system width from time representation. The important conclusion was that

7.2 Appendix B - Open Questions Index

The following unresolved issues recur across current docs, archive reports, and conversation evidence. They should be treated as review prompts, not as settled decisions.

Crosses unresolved or quarantined ownership areas. It is also not currently release-ready. Full CTest and broader release/trust proof remain visible debt outside the normal fast strict development gate. Refusal and defer semantics are part of determinism and replay guarantees. Inconsistent clients, shard drift, and audit failure. 12. Risks, Failure Modes, and What Future Chats Might Get Wrong. SDK locking was treated as mandatory. The warning was that allowing a newer Windows SDK to leak into old-floor builds can create symbol/runtime-loader failures and hidden dependency creep. Another tradeoff is between a clean theoretical structure and the current repo's transitional debt. The current ownership-root target is good, but the repo

still has cleanup work. The safe path is not broad movement by directory name; it is gated. The current repo should not be overstated as a complete runtime implementation of all v1 goals. Included specific risks about overclaiming implementation status. Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs. Use negative architecture rules to prevent drift. The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system. The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed. Distant machines become process. Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt. The unresolved detail is the exact representat

7.3 Appendix C - Source Notes Companion

The source notes companion records the file paths, evidence IDs, and source contributions used to write this book. It is separate so the main book can remain readable.

7.4 Appendix D - Topic Index

Major topics covered by this book include docs, packs, determinism, tooling, authority, contracts, identity, renderer, release, runtime, product, world, workbench, civilization, blocked. The standalone topic index expands these into chapter references and source-note groups.